(Frisch-Waugh-Lovell)′

Damian Clarke
Department of Economics, University of Exeter & University of Chile & MIPP
dclarke@fen.uchile.cl

Nicolás Paris Torres
Economics Department, University of Chile
nparis@fen.uchile.cl

Benjamín Villena-Roldán
Department of Economics, Universidad Andres Bello & LM^{2}C^2 & MIPP
benjamin@benjaminvillena.com

November 10, 2023

Abstract

We demonstrate that regression models can be estimated by working independently in a row-wise fashion. We document a simple procedure which allows for a wide class of econometric estimators to be implemented cumulatively, where, in the limit, estimators can be produced without ever storing more than a single line of data in a computer's memory. This result is useful in understanding the mechanics of many common regression models. These procedures can be used to speed up the computation of estimates computed via OLS, IV, Ridge regression, LASSO, Elastic Net, and Non-linear models including probit and logit, with all common modes of inference. This has implications for estimation and inference with 'big data', where memory constraints may imply that working with all data at once is particularly costly. We additionally show that even with moderately sized datasets, this method can considerably reduce computation time compared with traditional estimation routines.

Keywords: Big data, estimation, regression, matrix inversion.
JEL codes: C55, C61, C87.

Acknowledgements: We thank Richard Blundell, Samuel P. Engle and Jeffrey Wooldridge for their feedback and suggestions, and are grateful to Iván Gutierrez Martínez for excellent research assistance. The authors acknowledge MIPP for financial and institutional support. Clarke: University of Exeter, University of Chile, IZA, MIPP and CAGE. Contact email: dcc213@exeter.ac.uk. Nicolás Paris Torres: University of Chile. Contact email: nparis@fen.uchile.cl. Villena Rodán: Universidad Andrés Bello, MIPP, and LM^{2}C^2. Contact email: benjamin@benjaminvillena.com.

1 Introduction

The Frisch-Waugh-Lovell theorem is a canonical result in econometrics, and the foundation of many modern econometric estimation procedures. That a regression can be estimated by partitioning data column-wise is intuitive, and has a multitude of applications when brought to real data. Perhaps surprisingly, especially in a time where datasets are growing ever-larger and more decentralised, no such result has been highlighted when partitioning data *row*-wise. We address this in this paper.

We show that many regression models can be estimated by partitioning data in blocks of rows and that these partitions can be arbitrarily small or large. This implies that for a large class of regression models, there is no need for data ever to be stored in a single matrix, or ever stored in a computer's working memory. As well as the conceptual elegance of this result, we show that it can be of substantial use, especially when data is large. When data is so large that it escapes the working memory of a computer, this row-wise partitioning of data implies that estimation can proceed, with processing time simply scaling linearly with the number of observations. However, even where data is not too large to fit in a computer's working memory, we show that this result may offer a speed-up over standard commercial regression implementations, where in practice, processing times tend to not scale linearly with observations.

The basic intuition of this results is that many regression models require taking sums over cross-products of matrices of data such as a series of independent variables X of dimension $N \times K$. A clear example is the OLS estimator $\widehat{\beta}_{OLS} = (X'X)^{-1}X'y$. In practice, calculating $X'X$ requires summing over all observations $i \in \{1, \ldots, N\}$ the product of each value of each independent variable within observations i, but *does not* require cross-products taken across observations $j \neq i$. Similarly, $X'y$ requires that for a given observation i, the value of each of k independent variables x_{ki} be multiplied by y_i, but such a cross-multiplication is not required between observations. This implies that one can arrive very simply to aggregates such as $X'X$ and $X'y$ without ever reading an entire dataset into memory. Indeed, in the limit, one can calculate these quantities by reading a line of data for a single observation i at a time, iterating over all N, but never holding more than a single line of raw data in memory.[1] This may seem surprising given that regression models account fully for the interdependence between independent variables, but it is a base result of matrix algebra and the mechanics of multivariate regression.

What is more, a similar sequential procedure can be conducted for the variance of estimates of regression coefficients, as well as standard goodness-of-fit parameters such as the R-squared, implying that standard errors, confidence intervals, and any hypothesis tests can also be calculated exactly without ever loading all data in memory. We show that, a similar logic can be used for alternative models such as two-stage least

[1] A simple visualisation of this calculation is provided in Appendix A.

squares (2SLS), penalized regression models such as LASSO, Ridge and elastic net, and the estimation of probit and logit models using indirect least squares. Similar results can be derived for models where maximum likelihood (ML) estimation procedures are used. The methods discussed here are surprisingly flexible, also being feasible (and comparatively fast) with estimates which *prima facie* one may believe require loading larger portions of data in memory. For example, we show that similar *row*-wise procedures can be used for cluster-robust variance-covariance estimates without ever reading data on an entire group of observations at once.

In this paper we begin by establishing that linear regression models can be estimated row-wise, without ever opening the entire dataset. We define a "cumulative ordinary least squares" algorithm, which is exactly identical to OLS in both point estimates and standard errors, as well as any of the other basic statistics desired which are commonly reported following the estimation of OLS models. We then document that this result extends to alternative variance estimators and models. We discuss the computational implementation of such an estimator, noting that one could elect to split data into arbitrarily small or arbitrarily large partitions, though in practice, partitions should be sufficiently small such that they do not approach the limits of a computer's working memory.

In our research, we highlight an intriguing aspect that sheds light on the efficiency of regression estimation. While drawing comparisons to the Frisch-Waugh-Lovell theorem, we argue that understanding the possibility of estimating regression in blocks–with minimal information required to be saved between iterations–is of significant interest. The Frisch-Waugh-Lovell theorem, known for its column-wise approach, provides a technique to reduce the total dimension of K in calculations. In contrast, our paper introduces a 'transposed' version, presenting a row-wise result that allows for the reduction of the total dimension of N in calculations. Generally, and especially with the growing relevance of high-frequency and administrative datasets in economic research, N is substantially greater than K, meaning that reductions in N can lead to far greater computational savings than reductions in K. Notably, both the Frisch-Waugh-Lovell theorem and our paper are motivated by computational considerations, highlighting the shared emphasis on addressing computational challenges in econometrics (MacKinnon 2023b). However, beyond the computational elements of the paper, both of these results are theoretically elegant, and provide an understanding of the internal workings of regression models: one of the most commonly used tools of researchers in all empirical fields of economics.

This paper joins other studies from a range of settings which provide basic understanding of regression-based models (see for example Słoczyński (2022), Abadie (2003), Stefanski and Boos (2002), Gelbach (2016), Angrist (1998), Abadie, Athey, Imbens, and Wooldridge (2020), Solon, Haider, and Wooldridge (2015)). It also provides results which are potentially highly useful in cases where very large databases are

used in econometric analysis. Massive databases are increasingly common in econometric analyses (Varian 2014), but supercomputers are not always available to process big chunks of data. Indeed, MacKinnon (2023b) calls for consideration of computational issues with large datasets, noting that "[i]n recent years …many interesting datasets seem to be becoming larger more quickly than computers are becoming faster." While true, the results in this paper suggest that many of the processes of interest in econometrics can be implemented in a partition-wise fashion, implying that memory costs can be avoided. While an alternative solution to these issues is to simply gain access to super-computers or large server clusters, this solution may be infeasible for individuals with small research budgets or students, who nevertheless wish to use large datasets. These results can thus also be viewed as democratizing access to econometric tools. Finally, we note that these results can offer substantial speed-ups for clustered bootstrapping, joining a literature which considers the computational efficiency of bootstrap procedures, and clustering in particular (see eg Cameron et al. (2008), Roodman et al. (2019), MacKinnon (2023a)), as well as for the consideration of tuning parameters in regularised regression models.

This paper is structured as follows. In Section 2 we define the cumulative least squares procedure, showing its equivalence to standard estimation. We begin by showing how this estimator works in cases where estimation proceeds by OLS assuming homoscedasticity, and then document how it holds in a broad range of other estimation and inference procedures. Section 3 discusses the nature of the cumulative procedure, and considerations of optimal block sizes for estimation. In Section 4 we provide a number of illustrations of the performance of these methods compared to commonly-used commercial alternatives. This includes controlled tests where sample sizes and covariate numbers are varied and computational efficiency is compared, as well as an applied example based on a sample of census data and demographic surveys and models with a large number of fixed effects. In Section 5 we provide some additional discussion and conclusions.

2 Cumulative Least Squares

2.1 Cumulative Ordinary Least Squares

Suppose we wish to run a regression of a dependent variable y on a set of K covariates $x_1, x_2, ..., x_K$, using a series of $i = 1, \ldots, N$ observations. Thus, data can be viewed as a matrix or database of size $N \times K$ independent variables which we will denote X, as well as an $N \times 1$ vector Y for the dependent variable. Suppose also that computing the regression with all the data in memory is either infeasible or undesired due to memory constraints. The data can be partitioned row-wise in J arbitrarily defined portions, where each

portion, or block, is denoted j, and consists of $N_j = N/J$ observations.[2] The blocks are mutually exclusive and cover all observations such that $\sum_{j=1}^{J} N_j = N$. We use the notation X^j to denote block j of size N_j of the independent variables, and similarly Y^j is used to denote block j of the dependent variable.

Consider the OLS estimator of the parameter $\widehat{\beta}$. The standard OLS estimator can be written as follows:

$$\widehat{\beta}_{OLS} = (X'X)^{-1}X'Y \equiv \left(\begin{pmatrix} X^{1\prime} & X^{2\prime} & \cdots & X^{J\prime} \end{pmatrix} \begin{pmatrix} X^1 \\ X^2 \\ \vdots \\ X^J \end{pmatrix} \right)^{-1} \begin{pmatrix} X^{1\prime} & X^{2\prime} & \cdots & X^{J\prime} \end{pmatrix} \begin{pmatrix} Y^1 \\ Y^2 \\ \vdots \\ Y^J \end{pmatrix} \quad (1)$$

$$= \left(X^{1\prime}X^1 + X^{2\prime}X^2 + \cdots + X^{J\prime}X^J\right)^{-1}\left(X^{1\prime}Y^1 + X^{2\prime}Y^2 + \cdots + X^{J\prime}Y^J\right) \quad (2)$$

where in (1) the $K \times N$ matrix X' is re-expressed (identically) as a series of horizontally concatenated $K \times N_j$ matrices, and the $N \times K$ matrix X is similarly re-expressed as a series of vertically concatenated $N_j \times K$ matrices. The $N \times 1$ vector Y is also re-written as a series of vertically stacked sub-vectors of dimension $N_j \times 1$. Based on the properties of matrix multiplication, it can easily be seen that elements from each sub-matrix or vector will be interacted only with themselves, and no products are required across blocks. The re-expressed version of $\widehat{\beta}_{OLS}$ in (2) makes clear that $X'X$ can thus be re-written as the summation over the series of J matrices $X^{j\prime}X^j$ which are each of dimension $K \times K$, and a similar procedure can be followed for $X'Y$.

This suggests that a cumulative procedure can be followed, as laid out formally in Algorithm 1 below. Specifically, for ease of notation denote $X^{j\prime}X^j \equiv \Sigma_j$ and $X^{j\prime}Y^j \equiv \Upsilon_j$. Define as $\Sigma_{1\sim j}$ the summation $\Sigma_1 + \ldots \Sigma_j$, and $\Upsilon_{1\sim j} = \Upsilon_1 + \ldots + \Upsilon_j$. Thus, in estimating (2), initially a single block of data can considered, and the quantities Σ_1 and Υ_1 calculated. In the following step, a new block of data can be consulted, the quantities Σ_2 and Υ_2 calculated, and the preceding quantities summed to provide $\Sigma_{1\sim 2}$ and $\Upsilon_{1\sim 2}$. In following steps, accumulated quantities $\Sigma_{1\sim j-1}$ and $\Upsilon_{1\sim j-1}$ are received at the beginning of each stage, the Σ_j and Υ_j are calculated, and the step ends with $\Sigma_{1\sim j}$ and $\Upsilon_{1\sim j}$. A key element of this procedure is that in each stage, only a single block of data of size $N_j \times (K+1)$ needs to be read into memory, with the results stored in a single accumulated matrix and vector $\Sigma_{1\sim j}$ and $\Upsilon_{1\sim j}$. As Σ and Υ are of dimensions $K \times K$ and $K \times 1$ respectively, this makes clear that we simply need to keep track of small matrices in an ongoing fashion, and never house more than N_j observations in memory at a single time, where N_j can be an arbitrarily small value (in the limit, this could even be 1). The OLS estimate $\widehat{\beta}_{OLS}$ is *only* calculated by matrix inversion (or alternative procedures such as Gauss-Jordan elimination) once the full matrices $\Sigma_{1\sim J} \equiv X'X$ and $\Upsilon_{1\sim J} \equiv X'Y$ are calculated, implying that potentially costly matrix inversions are not

[2] To fix ideas, we will consider that N_j is common across all blocks. In Section 3 we will discuss optimal choices of N_j, not requiring that this number be equivalent across blocks.

required at every step.

The above process allows for point estimates to be recovered from independent partitions of the database. What's more, inference on regression parameters can be conducted in a similar partition-wise manner. Assuming homoscedasticity (alternative inference procedures are considered in Section 2.4), the well-known formula for the variance of OLS regression parameters is $\widehat{V}(\widehat{\beta}_{OLS}) = \widehat{\sigma}^2_U (X'X)^{-1}$. The quantity $(X'X)$ is already accumulated as laid out above. The second element of the variance is $\widehat{\sigma}^2_U \equiv \widehat{U}'\widehat{U}/(N-K)$, where the regression residuals $\widehat{U} = Y - X\widehat{\beta}_{OLS} = (I - X(X'X)^{-1}X')Y = M_X Y$, with M_X being the annihilator matrix, an idempotent matrix. Hence:

$$\widehat{U}'\widehat{U} = Y'M_X Y = Y'Y - Y'X(X'X)^{-1}X'Y, \tag{3}$$

which consists of three separate elements: $X'X$, $X'Y$ and its transpose, and $Y'Y$. The first two of these elements are already calculated iteratively in the estimation of point estimates as $\Sigma_{1\sim J}$ and $\Upsilon_{1\sim J}$. The only additional element required to calculate $\widehat{\sigma}^2_U$ is thus $Y'Y$, which can similarly be calculated in a cumulative manner in the same fashion as $X'X$ or $X'Y$ in (2): $Y'Y = (Y^{1\prime}Y^1 + Y^{2\prime}Y^2 + \cdots + Y^{J\prime}Y^J)$. As above, we will refer to $Y^{j\prime}Y^j \equiv \Psi_j$, and $\Psi_{1\sim j} = \Psi_1 + \ldots + \Psi_j$. Hence, calculating $Y'Y$ occurs iteratively, where at each step the accumulated $\Psi_{1\sim j-1}$ is the starting point, an additional block of Y of dimension $N_j \times 1$ is loaded, and the step ends with $\Psi_{1\sim j} = \Psi_{1\sim j-1} + \Psi_j$.[3]

Formally, the entire estimation process to arrive to exact OLS point estimates and standard errors is laid out in Algorithm 1. Note that given the information calculated in Algorithm 1, other standard regression statistics can be generated following estimation, including t-tests for each regression parameter against arbitrary null hypotheses, global F-tests of regressions, and R^2 or adjusted R^2 measures. For example, in order to compute the R^2, we can use the residual sum of squares (RSS), $U'U$ calculated above, and additionally require the total sum of squares (TSS), given that $R^2 = 1 - \frac{RSS}{TSS}$. The TSS is simply:

$$TSS = \sum_{i=1}^{N} \left(Y_i - \overline{Y}\right)^2 = \sum_{i=1}^{N} Y_i^2 - N\left(\frac{1}{N}\sum_{i=1}^{N} Y_i\right)^2 = \sum_{i=1}^{N} Y_i^2 - \frac{1}{N}\left(\sum_{i=1}^{N} Y_i\right)^2,$$

and both Y_i^2 and Y_i can be summed iteratively, with the only addition to the statistics already laid out above is the cumulative sum of Y squared.

[3] In Appendix D we note that this result can be documented in an alternative way, where rather than accumulating matrices Σ, Υ, and Ψ at each step, the estimate $\widehat{\beta}$ is directly updated. This result is based on the matrix inverse lemma (Woodbury 1950). However, given that this is less efficient than the cumulative procedure described here, we document this only as a curio.

Algorithm 1: Cumulative ordinary least squares

Inputs: Database consisting of (y,X), block size b.
Result: Point estimate $\widehat{\beta}_{OLS}$ and variance-covariance matrix $\widehat{V}(\widehat{\beta}_{OLS})$.

1. Set $i = 1$ and $j = b$. Load into memory partition of data covering y, X in observations i to j. ;
2. Calculate Σ_1, Υ_1, and Ψ_i ;
3. If observations i to j contain end of file, set $e = 1$, otherwise, set $e = 0$;

while $e \neq 1$ **do**

4. Replace $i = i + b$ and $j = j + b$. Load into memory partition of data covering Y, X in observations i to j. ;
5. Calculate Σ_j, Υ_j, and Ψ_j. ;
6. Calculate $\Sigma_{1\sim j} = \Sigma_{1\sim j-1} + \Sigma_j$, $\Upsilon_{1\sim j} = \Upsilon_{1\sim j-1} + \Upsilon_j$, and $\Psi_{1\sim j} = \Psi_{1\sim j-1} + \Psi_j$;
7. If observations i to j contain end of file, set $e = 1$. ;

end

8. Calculate

$$\widehat{\beta}_{OLS} = (\Sigma_{1\sim J})^{-1}\,\Upsilon_{1\sim J} \qquad \text{and} \qquad \widehat{V}\left(\widehat{\beta}_{OLS}\right) = \widehat{\sigma}^2 (X'X)^{-1},$$

where $\widehat{\sigma}^2 = \left[\Psi_{1\sim J} - \Upsilon'_{1\sim J}(\Sigma_{1\sim J})^{-1}\Upsilon_{1\sim J}\right]/(N-K)$.

2.2 Alternative Estimators

While the previous implementation allows for the generation of exact equivalents to OLS estimates and their standard errors (and derived statistics), this cumulative procedure can be applied far more widely. Indeed, the procedure can be used for *any* estimator which can be expressed as a sum of squares-based procedure, where the relevant database level processing requires sums over observation-level products. We document how the cumulative process works in a range of estimators below. We then document that similar logic can be used to arrive to estimates which are based upon other estimation techniques such as maximum likelihood.

Weighted Least Squares A simple extension to the procedure noted in section 2.1 is weighted least squares, where some diagonal weight matrix W is incorporated, such that the estimator is defined as:

$$\widehat{\beta}_{WLS} = (X'WX)^{-1}X'WY = \left(X^{1\prime}W^1X^1 + \cdots + X^{J\prime}W^JX^J\right)^{-1}\left(X^{1\prime}W^1Y^1 + \cdots + X^{J\prime}W^JY^J\right). \quad (4)$$

Here, it follows that an identical updating procedure can be implemented to that laid out in the case of OLS, however additionally, a variable w contains the weight associated with each observation. In this case, the cumulative estimation procedure consists of holding in memory a single block of data (Y_j, w_j, X_j) and generating matrix W_j, an $N_j \times N_j$ matrix with elements w_j on the principal diagonal. In the limit, if $N_j = 1$, the matrix W_j consists simply of the scalar w_j. Then, elements $X^{j\prime}W^jX^j$ and $X^{j\prime}W^jY^j$ are calculated,

and summed cumulatively, before in a final step the WLS estimator is calculated by matrix inversion or similar.

Instrumental Variables and Two-Stage Least Squares Estimators Both instrumental variables (IV) and Two-Stage Least Squares (2SLS) estimators can be similarly estimated in cumulative form. To see this, note that the IV estimator in a linear model is $\widehat{\beta}_{IV} = (X'Z)^{-1}Z'Y$ and the 2SLS estimator in a linear model is: $\widehat{\beta}_{2SLS} = \left(X'Z\left(Z'Z\right)^{-1}Z'X\right)^{-1}\left(X'Z\left(Z'Z\right)^{-1}Z'Y\right)$, where Z refers to an $N \times L$ dimensional vector of exogenous variables, with $L \geq K$, and in the case of IV, $L = K$. Thus, both $\widehat{\beta}_{IV}$ and $\widehat{\beta}_{2SLS}$ can be generated cumulatively following a similar procedure to (1), however in the case of $\widehat{\beta}_{IV}$ $X'X$ is substituted for $X'Z$, and $X'Y$ is substituted for $Z'Y$. In the case of 2SLS, an additional quantity $Z'Z$ must be calculated, though identically to $X'X$, this simply requires cross-products on all variables Z within each observation i, and as in (2), $Z'Z \equiv (Z^{1\prime}Z^1 + Z^{2\prime}Z^2 + \ldots + Z^{J\prime}Z^J)$. Once again, estimation can proceed in this case in a cumulative fashion, where in each block the quantities $X^{j\prime}Z^j$, $Z^{j\prime}Z^j$ and $Z^{j\prime}Y^j$ are calculated, summed cumulatively, and ultimately, the quantity $\widehat{\beta}_{2SLS}$ is calculated by matrix inversion and multiplication, or other standard procedures such as QR decomposition or single value decomposition.

Ridge, LASSO and Elastic Net Frequently in cases where big data is used in economic models, practitioners wish to perform some sort of regularisation. Fortunately, these cumulative procedures cross-over seamlessly to regularised models such as Ridge, LASSO and Elastic Net. Additionally, in each case, the process of accumulation is such that work with the full dataset of dimension $N \times K$ can be viewed as a first data processing step, and the selection of tuning parameters can be conducted as a second step, without ever returning to full data.

To see this, we first document the case of the Ridge regression, which given its use of the ℓ^2 norm for shrinkage is particularly simple expositionally. In the case of the Ridge regression, parameters are estimated as follows:

$$\widehat{\beta}_{Ridge} = \text{argmin}_{\beta}\left\{\left(\sum_{i=1}^{N}(Y_i - X_i'\beta)\right) + \lambda\sum_{j=1}^{K}\beta_j^2\right\},$$

where λ is a scalar tuning parameter determining the degree of shrinkage. This can equivalently be written as:

$$\widehat{\beta}_{Ridge} = (X'X + \lambda I)^{-1}X'Y \tag{5}$$

where I is an identity matrix of size K. Note that following the notation above, solving for $\widehat{\beta}_{Ridge}$ requires the quantity $\Sigma_{1\sim J}$, which we have documented can be calculated in a cumulative fashion, $\Upsilon_{1\sim J}$, which we have also documented can be calculated cumulatively, and an additional factor λI, which is independent of

the number observations. Hence, estimation in the case of Ridge is identical to that documented in OLS in section 2.1, with the only difference being that after accumulating $\Sigma_{1\sim J}$ and $\Upsilon_{1\sim J}$, but prior to solving for $\widehat{\beta}_{Ridge}$, an additional $K \times K$ matrix is summed to $\Sigma_{1\sim J}$. This is laid out formally in Algorithm 2.

Algorithm 2: Cumulative least squares for Ridge regression

Inputs: Database consisting of (y,X), block size b.
Result: Point estimate $\widehat{\beta}_{Ridge}$.

1. Set $i = 1$ and $j = b$. Load into memory partition of data covering y, X in observations $i$to j. ;
2. Calculate Σ_1 and Υ_1. ;
3. If observations i to j contain end of file, set $e = 1$, otherwise, set $e = 0$;

while $e \neq 1$ **do**

4. Replace $i = i + b$ and $j = j + b$. Load into memory partition of data covering Y, X in observations i to j. ;
5. Calculate Σ_j and Υ_j. ;
6. Calculate $\Sigma_{1\sim j} = \Sigma_{1\sim j-1} + \Sigma_j$ and $\Upsilon_{1\sim j} = \Upsilon_{1\sim j-1} + \Upsilon_j$. ;
7. If observations i to j contain end of file, set $e = 1$. ;

end

8. Select tuning parameter λ. Then calculate

$$\widehat{\beta}_{Ridge} = (\Sigma_{1\sim J} + \lambda I)^{-1}\Upsilon_{1\sim J}.$$

Similar procedures can be conducted in the case LASSO and Elastic net, where first data can be accumulated to form Σ_J and Υ_J and then, conditional on having processed the data of dimension $N \times K$ to a level of $K \times K$ (or $K \times 1$), and selecting a tuning parameter[4], estimates are calculated without ever returning to data at a level of $N \times K$ (or even $N_J \times K$). To see this, note that the Lasso and Elastic net equivalents of (5) are:

$$\begin{aligned}
\widehat{\beta}_{Lasso} &= \text{argmin}_\beta \left\{ \left(\sum_{i=1}^{N} (Y_i - X_i'\beta) \right) + \lambda ||\beta_j||_1 \right\}, \\
\widehat{\beta}_{ElasticNet} &= \text{argmin}_\beta \left\{ \left(\sum_{i=1}^{N} (Y_i - X_i'\beta) \right) + \lambda_1 ||\beta_j||_1 + \frac{\lambda_2}{2} ||\beta_j||_2^2 \right\},
\end{aligned}$$

where $||\cdot||_p$ refers to the ℓ^p norm, and in the case of the Elastic net, λ_1 and λ_2 refer to the strength of the Lasso and Ridge penalties respectively.

Although the lack of the exclusive ℓ^2 norm in Lasso and Ridge does not admit a simple least-squares solution as in (5), they nevertheless can both be simply resolved using cumulative procedures and a single (accumulatory) pass through N dimensional data. Specifically, this can be implemented via coordinate

[4] We note below that our procedures can similarly be used very efficiently for k-fold cross validation; see Section 2.3.

descent, a standard way of computing parameters in Lasso and Elastic Net (Fu 1998). To see this note that for a specific parameter β_j, the coordinate descent algorithm for estimation can be written for Lasso as:

$$\beta_j^{\text{new}} = \text{sign}\left(\beta_j^{\text{old}}\right) \max\left(|z_j| - \frac{\lambda}{N}, 0\right)$$

where β_j^{old} is the value of β_j at iteration the previous iteration, z_j is the jth element of the vector $z = X'(y - X\beta^{\text{old}})$ and sign$(\cdot)$ returns the sign of the argument. Noting that z can be re-expressed as $X'y - X'X\beta^{\text{old}}$ makes clear that the vector of parameters β_j can be estimated by first using the cumulative procedure laid out previously, and then working with $X'Y$ and $X'X$ in coordinate descent, without ever returning to the original data. A similar procedure can be used for Elastic Net given that in this case successive iterations of coordinate descent can be calculated as:

$$\beta_j^{\text{new}} = \frac{z_j}{1+\lambda_2}\left(\text{sign}\left(\beta_j^{\text{old}}\right) \max\left(|z_j| - \frac{\lambda_1}{1+\lambda_2}, 0\right)\right),$$

where z_j is the jth element of $z = X'(y - X\beta^{\text{old}}) + \beta^{\text{old}} \cdot \lambda_2$, and λ_1 and λ_2 are Lasso and Ridge regularization parameters. Again, given that z can be expressed as $X'y - X'X\beta^{\text{old}} + \beta^{\text{old}} \cdot \lambda_2$, estimation can proceed by, firstly, accumulating $\Sigma_{1\sim J}$ and $\Upsilon_{1\sim J}$ in a block-by-block or line-by-line fashion, and then implementing coordinate descent with, at most, matrices of dimension $K \times K$.

Binary Choice Models via Iteratively Reweighted Least Squares The previously defined estimators can be implemented in a single cumulative step, potentially offering substantial speed-ups compared to traditional estimators in cases where both cumulative and standard estimators are feasible, but where limits are close to met when all observations are housed in working memory (further discussion on relative performance of cumulative and naive procedures are provided in the following sections). In the case of Binary Choice Models such as probit and logit models, cumulative procedures can similarly be implemented which exactly replicate non-cumulative procedures while at the same time never housing more than a small number of observations in memory. However in these cases it is not possible to implement these estimators as single shot processes, but rather multiple passes through the N rows of data must be conducted. Thus, while these procedures provide feasible implementations of estimators when the entire dataset cannot be held in a computer's working memory, they are unlikely to be as fast as standard procedures when memory is not a limiting factor.

Nevertheless, to see that cumulative procedures can also be implemented in non-linear models, one alternative is to use Iteratively Reweighted Least Squares (IRLS). IRLS allows for the estimation of the parameters in non-linear models in a step-wise fashion, where at each step the updated parameter estimates

are based on a weighted least squares problem (Nelder and Wedderburn 1972; Green 1984). Based on this, cumulative procedures can be used to conduct least squares estimators in each iteration. Specifically, the IRLS procedure for binary outcome models consists of iteratively solving the following equation until β^{old} and β^{new} converge:

$$\widehat{\beta}^{\text{new}} = (X'WX)^{-1}X'WZ \quad \text{where} \quad Z = X\widehat{\beta}^{\text{old}} + W^{-1}(y-p). \tag{6}$$

Here y is an $N \times 1$ vector of outcome variables, and p is predicted value for each unit $p_i(X_i, \beta^{\text{old}})$ such that $y - p$ represents prediction residuals. W is an $N \times N$ diagonal weight matrix with diagonal elements consisting of $p_i(X_i, \beta^{\text{old}})(1 - p_i(X_i, \beta^{\text{old}}))$. In the case of probit models, for example $p(X_i, \beta) \equiv \phi(X_i\beta)$, while in the case of logit models, $p(X_i, \beta) = \ln(X_i\beta/(1 - X_i\beta))$. Given that we have previously shown that weighted least squares can be estimated in a cumulative fashion, this procedure thus allows for the estimation of non-linear models where in each step of (6) β^{old} is taken as an input (in the first iteration, $\beta^{\text{old}} = 0$), and instead of estimating the weighted least squares estimator in a block-wise fashion following (4), the transformed variable Z from (6) is used in place of Y.

Maximum Likelihood and other M-Estimators The use of cumulative procedures like those described above can similarly be employed to with other classes of M-estimators where estimation is based on iterative optimisation procedures provided that observations are assumed to be independently sampled.[5] To see this, consider maximum likelihood estimation implemented using the Newton-Raphson method. Estimation occurs iteratively, where at each stage the Hessian and Score matrices are evaluated based on the current iteration of β. Specifically, estimation occurs as follows:

$$\widehat{\beta}^{\text{new}} = \widehat{\beta}^{\text{old}} - \left[\frac{\partial^2 \ell(\beta)}{\partial\beta\partial\beta'}\right]^{-1}_{\beta=\beta^{\text{old}}} \left[\frac{\partial \ell(\beta)}{\partial\beta}\right]_{\beta=\beta^{\text{old}}}, \tag{7}$$

with the ML solution occurring when this equation converges.

When observations are independent, the Hessian and score matrices in ML are written as summations over observations i. For example, in the case of the logit regression:

$$\frac{\partial \ell(\beta)}{\partial\beta} = \sum_{i=1}^{N}\Big[y_i F(-X_i\beta) - (1-y_i)F(X_i\beta)\Big]X_i \qquad \frac{\partial^2 \ell(\beta)}{\partial\beta\partial\beta'} = -\sum_{i=1}^{N} f(X_i\beta)X_iX_i',$$

[5] In cases where sampling is not assumed to be independent, generalisations of this procedure could be followed, but likelihood functions, and hence blocks in the data in cumulative procedures, would need to permit this dependence. We discuss one such case where sampling is not assumed to be independent in Section 2.4 below.

where $F(\cdot)$ and $f(\cdot)$ are the logit cdf and pdf respectively.[6] This suggests a cumulative procedure can be employed where a block of arbitrary size N_j can be read in to memory and the Hessian and Score matrix can be calculated for this block j based on the values $\beta = \beta^{old}$. The summation for each matrix can be stored, and then a subsequent block of size N_j can be read in, the Hessian and Score matrices can be calculated, and added to the previous values. This process can be updated cumulatively until the end of the data is reached. Finally, a new value for β can be calculated as in (7), either providing the ML estimate if convergence has occurred, otherwise data will be read again, and another iteration of (7) calculated. In this case, as noted previously with IRLS, this procedure is feasible when large databases cannot be read into memory in their entirety, but is unlikely to be as fast as a standard ML procedures if the entire data *can* be stored in memory.

2.3 Grouped Estimation Procedures, Fixed Effect Estimators, Heterogeneity, and Cross-Validation

In the previous section, results were shown based on arbitrary divisions of the data into mutually exclusive blocks. All of the previous results hold if rather than groups of data being based on positions, groups of data are based on some particular indicator. Consider a variable G capturing membership in some particular group, with group levels $g \in \mathcal{G}$. Using notation X_g and Y_g to indicate realisations of X and Y respectively for observations where $G = g$, it is well known that the OLS estimate $\widehat{\beta}_{OLS}$ can be generated over groups as:

$$\widehat{\beta}_{OLS} = \left(\sum_{g\in\mathcal{G}} X_g' X_g\right)^{-1} \left(\sum_{g\in\mathcal{G}} X_g' Y_g\right) \tag{8}$$

What's more, as was the case previously, quantities $X_g' X_g$ and $X_g' Y_g$ can be built up cumulatively from arbitrarily small portions of data.

In practice, this is simply a group-level generalisation of the procedure described in Algorithm 1. As in Section 2.1, consider data broken down into J row-wise partitions, with each block denoted j and consisting of N_j observations. For a particular group $g \in \mathcal{G}$ define $X_g^{j\prime} X_g^j \equiv \Sigma_j^g$, and similarly, $X_g^{j\prime} Y_g^j \equiv \Upsilon_j^g$. If no observations for group g are present in block j, Σ_j^g is simply defined to be a null matrix $O_{K,K}$, and Υ_j^g a

[6] Similar examples can be easily provided for other common models estimated via ML. In the case of the probit regression, these functions are written as summations over i of the following form:

$$\begin{aligned}
\frac{\partial \ell(\beta)}{\partial \beta} &= \sum_{i=1}^{N} \left[y_i \frac{\phi(X_i\beta)}{\Phi(X_i\beta)} - (1-y_i)\frac{\phi(X_i\beta)}{1-\Phi(X_i\beta)} \right] X_i \\
\frac{\partial^2 \ell(\beta)}{\partial \beta \partial \beta'} &= -\sum_{i=1}^{N} \phi(X_i\beta) \left[y_i \frac{\phi(X_i\beta) + X_i\beta\Phi(X_i\beta)}{\Phi(X_i\beta)^2} - (1-y_i)\frac{\phi(X_i\beta) - X_i\beta(1-\Phi(X_i\beta))}{[1-\Phi(X_i\beta)]^2} \right] X_i X_i'.
\end{aligned}$$

where $\phi(\cdot)$ and $\Phi(\cdot)$ are the normal pdf and cdf respectively.

null vector $O_{K,1}$. As previously, $\Sigma^g_{1\sim j}$ refers to the summation $\Sigma^g_1 + \ldots \Sigma^g_j$, and $\Upsilon^g_{1\sim j} = \Upsilon^g_1 + \ldots + \Upsilon^g_j$. A group-level generalisation of Algorithm 1 is described in Algorithm 3 below.

Algorithm 3: Grouped cumulative ordinary least squares

Inputs: Database consisting of (y,X,G), block size b.
Result: Point estimate $\widehat{\beta}_{OLS}$ and variance-covariance matrix $\widehat{V}(\widehat{\beta}_{OLS})$. Aggregates $\Sigma^g_{1\sim J}$ and $\Upsilon^g_{1\sim J}$ $\forall g \in \mathcal{G}$.

1. Set $i = 1$ and $j = b$. Load into memory partition of data covering y, X, G in observations i to j. ;
for $g \in \mathcal{G}^1$ **do**
  2. Calculate Σ^g_1, Υ^g_1, and Ψ^g_i ;
3. If observations i to j contain end of file, set $e = 1$, otherwise, set $e = 0$;
while $e \neq 1$ **do**
  4. Replace $i = i + b$ and $j = j + b$. Load into memory partition of data covering Y, X, G in observations i to j. ;
  for $g \in \mathcal{G}^j$ **do**
    5. Calculate Σ^g_j, Υ^g_j, and Ψ^g_j. ;
    6. If $\exists\ \Sigma^g_{1\sim j-1}$ calculate $\Sigma^g_{1\sim j} = \Sigma^g_{1\sim j-1} + \Sigma^g_j$, $\Upsilon^g_{1\sim j} = \Upsilon^g_{1\sim j-1} + \Upsilon^g_j$, and $\Psi^g_{1\sim j} = \Psi^g_{1\sim j-1} + \Psi^g_j$, otherwise, initialise $\Sigma^g_{1\sim j} = \Sigma^g_j$, $\Upsilon^g_{1\sim j} = \Upsilon^g_j$, and $\Psi^g_{1\sim j} = \Psi^g_j$;
  7. If observations i to j contain end of file, set $e = 1$. ;
end

8. Calculate $\Sigma_{1\sim J} = \sum_{g\in\mathcal{G}} \Sigma^g_{1\sim J}$, $\Upsilon_{1\sim J} = \sum_{g\in\mathcal{G}} \Upsilon^g_{1\sim J}$, and $\Psi_{1\sim J} = \sum_{g\in\mathcal{G}} \Psi^g_{1\sim J}$. Then:

$$\widehat{\beta}_{OLS} = (\Sigma_{1\sim J})^{-1}\Upsilon_{1\sim J} \qquad \text{and} \qquad \widehat{V}(\widehat{\beta}_{OLS}) = \widehat{\sigma}^2\Sigma^{-1}_{1\sim J},$$

where $\widehat{\sigma}^2 = \left[\Psi_{1\sim J} - \Upsilon'_{1\sim J}(\Sigma_{1\sim J})^{-1}\Upsilon_{1\sim J}\right]/(N-K)$.

Heterogeneity An immediate implication of this group-level cumulative procedure is that instead of generating a single $K\times K$ matrix $\Sigma_{1\sim J}$ and $K\times 1$ vector $\Upsilon_{1\sim J}$, N_G versions of these matrices will be generated, where N_G refers to the distinct number of groups. Estimation of overall OLS parameters can then occur following (8), or any other grouped level estimator can similarly be generated. However, given that group level statistics $\Sigma^g_{1\sim J}$ and $\Upsilon^g_{1\sim J}$ are also generated, identical models for any sub-samples can then be generated nearly instantaneously, without ever returning to individual level data. This includes estimates for each specific group g, but also for aggregated groups, such as groups of states or groups of countries. In Section 2.4 we will return to show that this also offers substantial benefits for inference in cases of (blocked) bootstrap procedures.

Fixed Effect Estimators We can similarly use these group-level procedures to generate fixed-effect estimators, again without ever returning to individual level data. To see how fixed effect estimators can also

be estimated in a cumulative fashion, we will now double-index observations as Y_{ig} denoting individual i withing group g. We are interested in the estimating the parameter-vector on some independent variables X_{ig}, while controlling for time-invariant group fixed effects μ_g. The fixed effect estimator can generated as an OLS regression on within transformed data. Specifically, this consists of estimating:

$$\begin{aligned} Y_{ig} - \bar{Y}_g &= (X_{ig} - \bar{X}_g)\beta_{FE} + (\mu_g - \bar{\mu}_g) + u_{ig} - \bar{u}_g \\ \dot{Y}_{ig} &= \dot{X}_{ig}\beta_{FE} + \dot{u}_{ig} \end{aligned}$$

where $\dot{Y}_{ig}$ denotes the within transformation of Y, and similarly for other variables. The term u_{ig} is a time-varying stochastic error, and $\bar{Y}_g$ refers to group-level means. The fixed effect estimator is then written as below:

$$\widehat{\beta}_{FE} = \left(\dot{X}_{ig}'\dot{X}_{ig}\right)^{-1}\dot{X}_{ig}\dot{Y}_{ig} = \left(X_{ig}'X_{ig} - \bar{X}_g'\bar{X}_g\right)^{-1}\left(X_{ig}'Y_{ig} - \bar{X}_g'\bar{Y}_g\right). \tag{9}$$

The key insight on the right-hand side of (9) is that $(X_{ig}-\bar{X}_g)'(X_{ig}-\bar{X}_g) = (X_{ig}'X_{ig}-\bar{X}_g'\bar{X}_g)$. To see why this is the case, note that we can write $\bar{X}_g$ as: M_gX_g, where $M_g = I_g - 1_g(1_g'1_g)^{-1}1_g'$ is a group-specific demeaning operator, and 1_g a matrix which indicates membership to group g as a column of ones when the observation belongs to the group, and 0s otherwise. Note also that M_g is an idempotent matrix. Then, $\dot{X}_{ig}'\dot{X}_{ig} = (X_{ig} - \bar{X}_g)'(X_{ig} - \bar{X}_g) = [(I - M_g)X_{ig}]'(I - M_g)X_{ig}$, and the matrix $(I - M_g)$ is symmetric and idempotent. Thus, the preceding quantity can be written as $X_{ig}'(I - M_g)X_{ig} = X_{ig}'X_{ig} - \bar{X}_g'\bar{X}_g$ as required. Also note, that the $K \times K$ matrix $\bar{X}_g'\bar{X}_g$ can be generated from an underlying $K \times 1$ vector of group level means and the number of observations in each group. Specifically, refer to a group level vector of variable means as $\bar{x}_g \equiv (\bar{x}_{1g}\ \bar{x}_{2g}\ \cdots\ \bar{x}_{Kg})$. Then $\bar{X}_g'\bar{X}_g = N_g \times \bar{x}_g'\bar{x}_g$, where N_g is the number of observations in group g. Identical logic holds to show that $\dot{X}_{ig}\dot{Y}_{ig} = (X_{ig}'Y_{ig} - \bar{X}_g'\bar{Y}_g)$, and $\bar{X}_g'\bar{Y}_g$ can be generated from group level averages $\bar{x}_g$, a $K \times 1$ vector, and scalar $\bar{y}_g$.

Given this, implementing fixed effect models using grouped data generated in a cumulative fashion is a straightforward extension of Algorithm 3. From (9), elements $X_{ig}'X_{ig}$ and $X_{ig}'Y_{ig}$ have already been calculated cumulatively for all g. The remaining step is to calculate $\bar{X}_g'\bar{X}_g$ and $\bar{X}_g'\bar{Y}_g$ which only requires group-level variable means. From this $K \times K$ matrices $\dot{X}_{ig}'\dot{X}_{ig}$ and $K \times 1$ vector $\dot{X}_{ig}'\dot{Y}_{ig}$ can be generated, and the fixed effect estimator (9) can be calculated as:

$$\widehat{\beta}_{FE} = \left(\sum_{g\in\mathcal{G}}\dot{X}_{ig}'\dot{X}_{ig}\right)^{-1}\left(\sum_{g\in\mathcal{G}}\dot{X}_{ig}\dot{Y}_{ig}\right)$$

Similar cumulative procedures can be followed for two-way fixed effect models using the double within-transformation (Baltagi 2001; Wooldridge 2021). For example for balanced panels over group g and time

t, two way transformations $\ddot{X}_{gt} = X_{gt} - \bar{X}_g - \bar{X}_t + \bar{X}$ and $\ddot{Y}_{gt} = Y_{gt} - \bar{Y}_g - \bar{Y}_t + \bar{Y}$ can be calculated, and similar procedures followed as in the fixed effect case.[7]

Returning Fixed Effects Generally, when fixed effect models are implemented, the interest is in estimating the coefficients and standard errors on time-varying variables, and hence a fixed effect estimator like (9) is appropriate. However, in cases where estimates and standard errors on fixed effects themselves are also desired, cumulative least squares procedures offer a particularly efficient way to generate these estimates.

To see this, note that in the case of mutually exclusive fixed effects, we can write:

$$X'X = \begin{pmatrix} \sum_{i=1}^N x_{1i}x_{1i} & \cdots & \sum_{i=1}^N x_{1i}x_{Ki} & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ \sum_{i=1}^N x_{Ki}x_{1i} & \cdots & \sum_{i=1}^N x_{Ki}x_{Ki} & 0 & \cdots & 0 \\ \bar{x}_{1,g_1} & \cdots & \bar{x}_{K,g_1} & N_{g_1} & \cdots & 0 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ \bar{x}_{1,g_K} & \cdots & \bar{x}_{K,g_K} & 0 & \cdots & N_{g_K} \end{pmatrix} \quad X'Y = \begin{pmatrix} \sum_{i=1}^N x_{1i}y_i \\ \vdots \\ \sum_{i=1}^N x_{Ki}y_i \\ \bar{y}_{g_1} \\ \vdots \\ \bar{y}_{g_K} \end{pmatrix},$$

where here we assume data is ordered such that first time-varying variables are included in X, and then group fixed effects. In this case, the resulting matrix $X'X$ simply consists of the $K \times K$ matrix $\Sigma_{1\sim J}$ in the top-left corner (where here K refers to time-varying variables, a $N_G \times K$ matrix of group means in the bottom left corner, the matrix O_{K,N_G} in the top right-hand corner, and a $N_G \times N_G$ diagonal matrix containing the number of observations in each group on the main diagonal. Similarly, $X'Y$ simply consists of the vector $\Upsilon_{1\sim J}$ in positions 1 to K, and then N_G group-level means below. In this case, the only required information beyond elements already stored in standard cumulative procedures ($\Sigma_{1\sim J}$ and $\Upsilon_{1\sim J}$), are group level means and observation numbers, which can be trivially estimated cumulatively. This thus suggests that fixed effect estimators can be estimated directly and efficiently *including all fixed effects* in a sequential procedure.

Cross-Validation In Section 2.2 we noted that cumulative procedures could be used models such as Ridge, LASSO and elastic net, were commonly tuning parameters are chosen. Often, such tuning parameters are chosen through k-fold cross-validation (see, *eg* Wu and Wang (2020)). We showed previously that the

[7] This results follows from the case of single demeaning. However, here observations i are observed over groups g and time t, and both group and time fixed effects need to be removed. Noting that we can now define the double-demeaning operation as $M_{gt} = [I_{gt} - 1_g(1_g'1_g)^{-1}1_g' - 1_t(1_t'1)^{-1}1' + 1(1'1)^{-1}1']$, and hence write $\ddot{X}'\ddot{X}$ and $\ddot{X}$ as $M_{gt}X_{gt}$, then $\ddot{X}'\ddot{X} = X_{gt}'M_{gt}'M_{gt}X_{gt}$. However, M_{gt} is idempotent, and so $\ddot{X}'\ddot{X} = (X_{gt}'X_{gt} - \bar{X}_g'\bar{X}_g - \bar{X}_g'\bar{X}_g + \bar{X}'\bar{X})$. This then suggests a simple and feasible process for concentrating out two-way (or higher order) fixed effects by grouping aggregates $\Sigma_{1\sim J}$ and $\Upsilon_{1\sim J}$ over g and t, and calculating group-specific, time-specific, and overall means, which can then be used to estimate $\widehat{\beta}_{FE}$ after processing all data.

tuning parameter λ in these models can be chosen after accumulating matrices $X'X$ and $X'Y$ (see for example the case of Ridge regression in (5)). If we follow Algorithm 3, where the group variable is simply a discrete uniform random variable taking values between 1 and k, resulting matrices $X_1'X_1, \cdots, X_K'X_K$, and $X_1'Y_1, \cdots, X_K'Y_K$, can be used for k-fold cross validation in an efficient way.

To see this, note that cross validation consists of a procedure where for a tuning parameter λ, a specific group g is held out, and coefficients $\widehat{\beta}_{-g,Ridge}(\lambda)$ estimated using the remaining groups. Within group g, the Mean Squared Error associated with this parameter is then calculated as $MSE = \frac{1}{N_g}||y_g - X_g\widehat{\beta}_{-g,Ridge}(\lambda)||^2$. A similar procedure is then conducted for each of the N_G groups, and the MSE associated with λ is calculated as the sum of the group-specific MSEs. Note that this quantity $\frac{1}{N_g}||y_g - X_g\widehat{\beta}_{-g,Ridge}(\lambda)||^2$ can be rewritten as $\frac{1}{N_g}\left[(y_g - X_g\widehat{\beta}_{-g,Ridge}(\lambda))'(y_g - X_g\widehat{\beta}_{-g,Ridge}(\lambda))\right] = y_g'y_g - 2\widehat{\beta}_{-g,Ridge}(\lambda)'X_g'Y_g + \widehat{\beta}_{-g,Ridge}(\lambda)'X_g'X_g\widehat{\beta}_{-g,Ridge}(\lambda)$. Each of the quantities $X_g'Y_g$, $X_g'X_g$ and $Y_g'Y_g$ are already calculated in a cumulative fashion, implying that the MSE for a given lambda can be calculated entirely from cumulatively calculated aggregates, and MSE-optimal tuning parameters chosen as the value of λ which minimises this MSE.

2.4 Alternative Inference Procedures

In Section 2.1 we documented that inference could be conducted in a cumulative fashion in the same way as point estimates, and this required no other special procedures, apart from the accumulation of $\Psi_{1\sim J}$, which is needed to calculate the variance-covariance matrix but not parameter estimates. This can all be done in a single pass through blocks of the data. However, this relies on a homoscedasticity assumption. Here we discuss how inference can proceed in alternative settings.

2.4.1 Heteroscedasticity Robust Standard Errors

In cases where heteroscedasticity-robust standard errors are desired, the well-known heteroscedasticity-robust estimator can be implemented cumulatively. The HC1 variance estimator for OLS is written as:

$$\widehat{V}(\widehat{\beta}_{OLS})_{HC1} = \frac{N}{N-K}(X'X)^{-1}\left[\sum_{i=1}^{N}\widehat{u}_i^2 x_i'x_i\right](X'X)^{-1}$$

From Section 2.1, we already know that $X'X = \Sigma_{1\sim J}$ can be generated cumulatively. Similarly, both K and N can be read trivially from data. If $\widehat{u}_i^2$ is known, the central component $\sum_{i=1}^{N}\widehat{u}_i^2 x_i'x_i$ could be calculated cumulatively: this value, which we will refer to as Ω could be initialised as a null matrix $O_{K,K}$, and in each block of the dataset when an observation i is read in, the quantity $\widehat{u}_i^2 x_i'x_i$ calculated, and summed

to all previous values, as laid out in the Algorithm 4 below. As above, we will define $\Omega_j \equiv \sum_{i \in j} \widehat{u}_i^2 x_i' x_i$, and $\Omega_{1 \sim j} \equiv \Omega_1 + \cdots \Omega_j$.

The issue here however is that when the data is first loaded in blocks, we cannot calculate $\widehat{u}_i = (Y_i - X_i \widehat{\beta}_{OLS})$, as this requires $\widehat{\beta}_{OLS}$, which is not known until an entire pass through the data has been completed. Thus, while heteroskedasticity robust estimates can be calculated in a cumulative fashion, this requires the data be read in a cumulative fashion a second time. In particular, first Algorithm 1 should be run to calculate $\widehat{\beta}_{OLS}$, and then Algorithm 4 be run to $\widehat{\beta}_{OLS}$ as an input. However, apart from having to return to read the data, there is no particular memory restriction which implies that this procedure will not be feasible. The only addition is a single accumulated $K \times K$ matrix $\Omega_{1 \sim J}$. Similar procedures can be conducted for IV and other estimators.

Algorithm 4: Cumulative Estimation of Heteroscedasticity-Robust Variance-Covariance Matrix

Inputs: Database consisting of (y,X), block size b. Point estimate $\widehat{\beta}_{OLS}$, and $\Sigma_{1 \sim J}$ from Algorithm 1.
Result: Variance-covariance matrix $\widehat{V}(\widehat{\beta}_{OLS})_{HC1}$.

1. Set $i = 1$ and $j = b$. Load into memory partition of data covering y, X in observations i to j ;
2. Calculate Ω_1 ;
3. If observations i to j contain end of file, set $e = 1$, otherwise, set $e = 0$;

while $e \neq 1$ **do**

4. Replace $i = i + b$ and $j = j + b$. Load into memory partition of data covering Y, X in observations i to j. ;
5. Calculate Ω_j. ;
6. Calculate $\Omega_{1 \sim j} = \Omega_{1 \sim j-1} + \Omega_j$. ;
7. If observations i to j contain end of file, set $e = 1$. ;

end

8. Calculate: $\widehat{V}(\widehat{\beta}_{OLS})_{HC1} = \frac{N}{N-K} \Sigma_{1 \sim J}^{-1} \Omega_{1 \sim J} \Sigma_{1 \sim J}^{-1}$.

2.4.2 Cluster-Robust Variance Covariance Matrix

Similarly, in the case of standard closed-form cluster-robust variance-covariance estimators, a second pass through of the data is required to calculate cumulative standard errors.[8] In this case, slightly more information must be stored, namely an additional vector of size K for each of the N_G groups over which clustering occurs, but unless both K and N_G are exceedingly large, this should not generate a problem for the feasibility of these procedures. Perhaps somewhat surprisingly, while clustered variance-covariance matrices

[8] In Section 2.4.3 we lay out an extremely efficient clustered bootstrap procedure in which it is not necessary to return to individual-level data.

account for dependence among observations, it is never necessary for data for an entire cluster to be housed in a computer's working memory in order to cluster standard errors by group.

To see this, note that the standard cluster-robust variance-covariance estimator is written as follows.

$$\widehat{V}(\widehat{\beta}_{OLS})_c = \frac{N-1}{N-K}\frac{N_G}{N_G-1}(X'X)^{-1}\left[\sum_{g\in\mathcal{G}} X_g'(\widehat{u}_g\widehat{u}_g')X_g\right](X'X)^{-1}$$

As previously g refers to groups over which clustered standard errors are desired, and N_G refers to the total number of groups. As in the case of the HC1 estimator, observation, group, and covariate quantities can be easily read in a cumulative fashion from data, and $X'X$ is similarly calculated cumulatively. However, here we additionally require the quantity $\Omega_g \equiv \sum_{g\in\mathcal{G}} X_g'(\widehat{u}_g\widehat{u}_g')X_g$. For expositional clarity, note that $\sum_{g\in\mathcal{G}} X_g'(\widehat{u}_g\widehat{u}_g')X_g = \sum_{g\in\mathcal{G}}(X_g'\widehat{u}_g)(\widehat{u}_g'X_g)$. Matrix X_g' is an $K \times N_g$ matrix, while $\widehat{u}_g$ is an $N_g \times 1$ vector of regression residuals. Thus, the matrix $X_g'\widehat{u}_g$ is a $K \times 1$ vector, while its transpose $\widehat{u}_g'X_g$ is $1 \times K$. Additionally, note that $X_g'\widehat{u}_g$ is generated by multiplying the observations *of each observation* with its own residual, and so can be generated cumulatively. Thus, as previously, if data is arbitrarily divided into J blocks, the quantity $X_g'\widehat{u}_g = X_g^{1\prime}\widehat{u}_g^1 + \ldots + X_g^{J\prime}\widehat{u}_g^J$ can be generated cumulatively by first calculating $X_g^{j\prime}\widehat{u}_g^j$ for each group present within each block j, then summing over all j, and finally using this quantity to calculate the overall quantity Ω_g.[9] This procedure is laid out formally in Algorithm 5 below, where as before, $X_g'\widehat{u}_{g,1\sim j}$ refers to the summation of $X_g^{1\prime}\widehat{u}_g^1 + \cdots + X_g^{j\prime}\widehat{u}_g^j$.

2.4.3 An Efficient Bootstrap Algorithm for Clustering

While the clustered procedure described in Section is feasible and permits for the exact calculation of analytic cluster-robust variance covariance matrices, it requires opening the data two times: the first to calculate the parameter estimates, and the second to calculate the standard errors which requires residuals $\widehat{u}_g$. However, given the results from Section 2.3, if one wishes to generate a clustered standard error by bootstrapping, this can be done in a single pass through the data, and additionally bootstrap replicates can be conducted extremely quickly, and indeed orders of magnitude more quickly than in standard clustered bootstrap procedures. To see why note that the parameter estimate of interest can be generated as in (8). Also note that from Algorithm 3, that cumulative procedures are used to generate $K \times K$ matrices for each group $\Sigma_{1\sim J}^g$, as well as group-specific $K \times 1$ vectors $\Upsilon_{1\sim J}^g$.

This implies that we can generate resampled versions of (8) by simply resampling with replacement N_G

[9] It is important to note that this procedure requires working with the $K \times 1$ vector $X_g^{j\prime}\widehat{u}_g^j$ at each step. It is *not* possible to calculate Ω_g at each step, but rather, we must accumulate $X_g^{j\prime}\widehat{u}_g^j$ and only then calculate Ω_g.

Algorithm 5: Cumulative Estimation of Cluster-Robust Variance-Covariance Matrix

Inputs: Database consisting of (Y,X,G), block size b. Point estimate $\widehat{\beta}_{OLS}$, and $\Sigma^{g}_{1\sim J}$ from Algorithm 3.
Result: Variance-covariance matrix $\widehat{V}(\widehat{\beta}_{OLS})_C$.

1. Set $i = 1$ and $j = b$. Load into memory partition of data covering G, X in observations i to j ;
for $g \in \mathcal{G}^1$ **do**
  2. Calculate $X_g^{1\prime}\widehat{u}_g^1$;
3. If observations i to j contain end of file, set $e = 1$, otherwise, set $e = 0$;
while $e \neq 1$ **do**
  4. Replace $i = i + b$ and $j = j + b$. Load into memory partition of data covering X, G in observations i to j. ;
  for $g \in \mathcal{G}^j$ **do**
    5. Calculate $X_g^{j\prime}\widehat{u}_g^j$. ;
    6. If $\exists\ X_g'\widehat{u}_{g,1\sim j-1}$ calculate $X_g'\widehat{u}_{g,1\sim j} = X_g'\widehat{u}_{g,1\sim j-1} + X_g^{j\prime}\widehat{u}_g^j$, otherwise initialise $X_g'\widehat{u}_{g,1\sim j} = X_g^{j\prime}\widehat{u}_g^j$. ;
  7. If observations i to j contain end of file, set $e = 1$. ;
end

8. Calculate

$$\widehat{V}(\widehat{\beta}_{OLS})_c = \frac{N-1}{N-K}\frac{N_G}{N_G-1}(\Sigma_{1\sim J})^{-1}\left[\sum_{g\in\mathcal{G}}(X_g'\widehat{u}_{g,1\sim J})(X_g'\widehat{u}_{g,1\sim j})'\right](\Sigma_{1\sim J})^{-1}$$

.

pairs of matrices $\Sigma^{g}_{1\sim J}$, $\Upsilon^{g}_{1\sim J}$, and calculating a resampled estimator $\widehat{\beta}^{b*}$ as follows:

$$\widehat{\beta}^{*} = \left(\sum_{g^*\in\mathcal{G}^*}\Sigma^{g*}_{1\sim J}\right)^{-1}\left(\sum_{g^*\in\mathcal{G}^*}\Upsilon^{g*}_{1\sim J}\right), \tag{10}$$

where $\Sigma^{g*}_{1\sim J}$ refers to resampled matrix $\Sigma^{g}_{1\sim J}$. When clusters are large, such as individuals within states of countries, resampling aggregated matrices to form bootstrap resamples $\widehat{\beta}^{*}$ will be orders of magnitudes faster than resampling clusters of data. This suggests a potentially substantially faster bootstrap estimate for the cluster robust variance for the parameter vector $\widehat{\beta}$. This consists of generating a large number B of resampled estimates (10), which can be used to calculate the bootstrap CRVE for $\widehat{\beta}$ as: $V(\widehat{\beta})_{CRVE} = V(\widehat{\beta}^{*}) = \frac{1}{B}\sum_{b=1}^{B}(\widehat{\beta}^{*}_b - E[\widehat{\beta}^{*}_b])^2$.

3 Optimal Implementation

Whether implementing cumulative or standard algorithms, identical calculations are required to be made, given that cross products are required between each element of X and between X and Y for each observation i. Indeed, cumulative algorithms require *strictly more* calculations than standard algorithms. To see this, consider the case of OLS. To calculate coefficients in OLS, X' must be multiplied with X, implying computational time of order $\mathcal{O}(NK^2)$. Additionally, X' must be multiplied with Y, implying computational time of order $\mathcal{O}(NK)$. Finally, resolving the linear system $X'X\beta = X'Y$ involves time $\mathcal{O}(K^3)$ via Gauss-Jordan elimination. In the case of cumulative algorithms, identical procedures are required, and additionally, at each step two $K \times K$ matrices Σ_j and $\Sigma_{1\sim j-1}$ must be summed, which is of computational time $\mathcal{O}(K^2)$, and similarly, two $K \times 1$ vectors Σ_j and $\Sigma_{1\sim j-1}$ must be summed, involving time $\mathcal{O}(K)$. In general, $N >> K$, implying that $\mathcal{O}(NK^2)$ will dominate in both cases.

Nevertheless, if all computational procedures scale linearly in the number of observations, no gains will be made by implementing cumulative routines in place of their standard counterparts. However, computational routines clearly do not scale linearly with sample size indefinitely. To see this, it is sufficient to consider two cases: one where N is such that observations can be housed in a computer's working memory, and another where N exceeds the capacity of a computer's memory. In the prior case, the calculation time will be finite, while in the latter case calculation will be impossible, and hence time will be infinite. In this section we will discuss the optimal implementation where optimality refers to the block size which minimises calculation time. Given that in the limit cumulative procedures converge to OLS if a block size of N is chosen, we consider only the optimal choice of block size for cumulative procedures. We return to these issues empirically in Section 4.

As above, the entire cumulative algorithm for OLS requires a number of well-defined steps. In total, matrix multiplication between X' and X is $\mathcal{O}(NK^2)$, and between X' and Y is $\mathcal{O}(NK)$. Final resolution of the parameters is $\mathcal{O}(K^3)$. Additionally, within each block j a series of element-by-element summations must occur to accumulate $\Sigma_{1\sim j}$ and $\Upsilon_{1\sim j}$. In each step these are are of order $\mathcal{O}(K^2)$ and $\mathcal{O}(K)$ respectively. Given that there are J such blocks, and in the first block it is not necessary to accumulate $\Sigma_{1\sim 1}$ and $\Upsilon_{1\sim 1}$, these calculations are of computational time $\mathcal{O}(K^2(J-1))$ and $\mathcal{O}(K(J-1))$. Thus, total computational time of the algorithm is of the order:

$$\mathcal{O}(NK^2) + \mathcal{O}(NK) + \mathcal{O}(K^3) + \mathcal{O}(K^2(J-1)) + \mathcal{O}(K(J-1)). \tag{11}$$

Here it is clear that if a single block is chosen, and hence $J = 1$, then $\mathcal{O}(K^2(J-1)) + \mathcal{O}(K(J-1)) = 0$ and the cumulative algorithm collapses to OLS.

To consider the optimal block size, we will consider separately three elements of (11). A first element, corresponding to the first two terms in (11) and denoted $L(N,K)$ is the procedure of loading data and multiplying matrices required to arrive to $X'X$ and $X'Y$. We write this function as $L(N,K) = l(NK^2 + NK)$. A second element, corresponding to the third term in (11), consists of generating estimates $\widehat{\beta}$ once provided with $X'X$ and $X'Y$, and is written as $S(K) = s(K^3)$. And finally, an accumulation procedure, denoted $C(J,K) = c(K^2(J-1)+K(J-1))$, consisting of the final two terms where matrices are summed in a cumulative fashion.

Note that given that $N = JN_j$, the first term can be re-expressed as s $L(J,K) = l(JN_jK^2 + JN_jK)$. For the sake of simplicity, given that N_j is determined by J, below we omit N_j terms as implicit in $l(\cdot)$. For a given K, The total time to compute the cumulative least squares algorithm can thus be written as:

$$T_c(J;K) = L(J,K) + S(K) + C(J,K).$$

Hence, the optimal number of partitions of data J should solve the problem:

$$\min_J \Big(L(J,K) + S(K) + C(J,K) \Big) \text{ subject to } 0 < J \leq N.$$

For an interior solution, this suggests that optimal number of blocks considered should satisfy the following first order condition:

$$\begin{aligned} &\frac{\partial L(J,K)}{\partial J} + \frac{\partial C(J,K)}{\partial J} = 0 \\ \Rightarrow &\frac{\partial L(J,K)}{\partial J} = -\frac{\partial C(J,K)}{\partial J} \end{aligned} \tag{12}$$

Note that here given that regardless of the block size chosen, the same final matrix inversion is required, for a given K optimality does not depend on $S(\cdot)$, as reflected in (12). This suggests the logical conclusion that an optimal block size should be chosen which equates the marginal cost of an summing an additional set of matrices across blocks, $\partial C(J,K)/\partial J$, with the marginal benefit coming from loading smaller partitions of the data into memory to calculate $X'X$ and $X'Y$.

Understanding the optimal block size for conducting cumulative least squares thus requires understanding the nature of functions $C(J,K)$ and $L(J,K)$. The precise nature of these two functions is likely highly dependent upon a particular computational environment (both software and hardware), nevertheless, we can suggest a number of key conjectures. Firstly, it is clear that for a given K, $C(J,K)$ will, abstracting from other elements, be linear in J. To see this, note that for each additional block, we simply require the summation of an additional identically sized $K \times K$ and $K \times 1$ matrix. Thus, moving from j to $j+1$ requires adding

one set of summations, while moving from $j+1$ to $j+2$ requires adding an identical set of summations, and so calculation time will scale linearly in block sizes. Secondly, for a given K, $L(J,K)$ seems unlikely to be linear in J. Rather, this value is highly dependent on a particular computational environment. Note that in general, when a computer's RAM usage is high a number of internal processes such as paging occur such that loading data into memory becomes increasingly slow as the size of a database increases. Thus, when a sample approaches the limit of a computer's RAM, the marginal benefit of increasing the number of blocks of data is high given that it avoids substantial slowdowns inherent in computational architecture. However, if a computer's RAM usage is low, the marginal benefit of increasing the number of blocks approaches zero, given that no such slowdown in data loading occurs, and the total computation time $L(J,K)$ is independent of J. Thus, at very high values of J, for example w.here J approaches the total number of observations, the marginal benefit of increasing $L(J,K)$ is likely essentially zero given that no memory slowdown occurs owing to the storage of large amounts of data in memory. However, at high low values of J, if data is large enough to result in memory slowdowns, the benefit of increasing J is substantial. On the other hand, the costs of increasing J, $C(J,K)$ are constant in J.

This suggests a number of general results. Firstly, if one is working with large datasets and memory is not unlimited, it is likely the case that smaller blocks of data should be preferred given that memory slowdowns can be avoided. If data does not fit in memory, this argument holds with certainty, given that $\partial L(J,K)/\partial J|_{J_{min}} = \infty$, where J_{min} refers to the point at which it becomes feasible to hold data in memory. However, if RAM limits are binding with N, the optimal solution is likely not to increase the number of blocks to the maximum theoretical limit ($J = N$), given that at low block sizes no memory slowdown will be observed, but a constant cost increase is observed in terms of sums across blocks. What's more, these results suggest that there is no gain from varying the block size across the sample, but rather that a single value of N_J should be chosen as that which satisfies (12). Finally, as the number of covariates increases, it seems likely that fewer blocks should be preferred, given that the cost of adding marginal blocks increases in K. Precise optima will vary across computers and configurations, and are thus specific to particular contexts. In the following section we will document specific examples which point to a Goldilocks principle of choosing blocks neither too big nor too small, and, fortunately, suggest that computation time is quite flat over a large range of blocks, provided that extreme situations are not encountered.

4 Illustrations

In this section we document two examples to illustrate the performance of cumulative procedures in practice. A first example is based on simulated data where we maintain fixed computational resources and vary key

parameters of the data (namely the number of observations and the number of variables). And a second example is based on real data, where we document the performance of cumulative versus standard estimation procedures in a range of computational environments and with various methods of estimation.

4.1 Simulated Data

To demonstrate the relative performance of the cumulative algorithm compared with a standard regression implementation, we test the time to complete calculations under controlled conditions. Specifically, we compare the time it takes to run an OLS regression using cumulative and standard estimation routines based on the same data. We conduct these tests on a server with 1GB of dedicated RAM and no outside processes running to ensure comparability across estimation times.[10] We consider a range of observation numbers and independent variables, and, in the case of the cumulative algorithm, also document times under a range of block sizes. In each case, the time completed consists of identical procedures: namely, in the case of the cumulative algorithm it is the time to import all blocks of data, calculate the necessary block-specific quantities, and finally return the regression estimates, standard errors and R-squared. And in the case of a 'standard' regression implementation, the time simply refers to the time to open the data from the disk and estimate the OLS regression using canned software.

The test procedure thus consists of generation of data of the following general form:

$$y = X\beta + u,$$

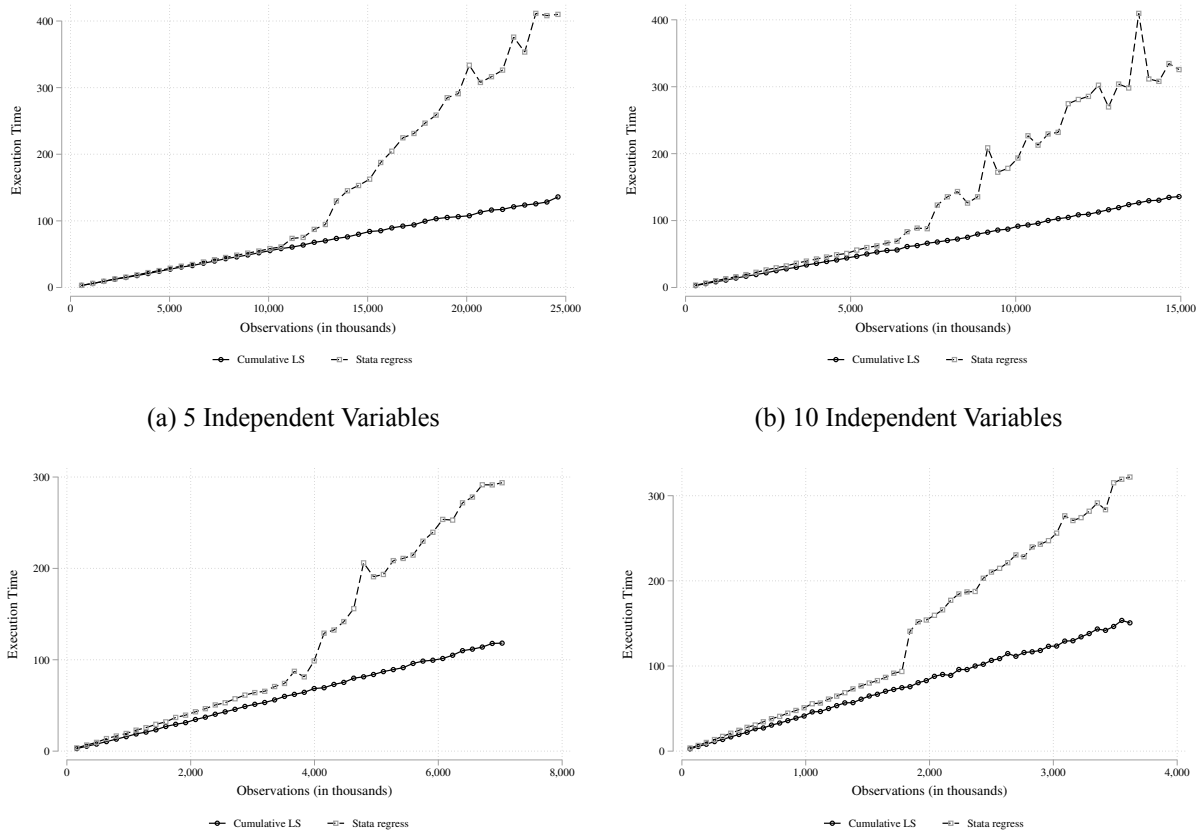
where X is an $N \times K$ matrix of simulated data consisting of a constant and $K-1$ uniformly distributed variables, $u \sim \mathcal{N}(0,3)$ is a simulated $N \times 1$ error term, and β is a $K \times 1$ vector of parameters. Here we consider processing times varying K, N and the block size, N_j, where in each case X and y are treated as inputs, u as unobservable, and β as a vector of parameters to estimate.

Tests are conducted using a recent version of Stata (specifically, Stata version 16), where the cumulative algorithm is written principally in Stata's matrix language Mata. Regression is conducted using Stata's native "regress" command. Initially, a single core version of Stata is used (Stata SE), however relative performance is shown to follow qualitatively similar patterns when a multiple processor version of Stata is used (Stata MP). In Section 4.2 we consider a range of alternative estimation procedures and models.

Processing times for estimation of cumulative algorithms versus standard regression software are documented in Figure 1. Each panel presents processing times for a particular number of simulated independent

[10] This is a commercially available Virtual Private Server with a 4 core CPU running a Linux-based operating system. All data is stored on the server on a solid state drive.

Figure 1: **Sample size and execution time of commercial routine versus cumulative method**



(a) 5 Independent Variables

(b) 10 Independent Variables

(c) 20 Independent Variables

(d) 50 Independent Variables

Notes: All times refer to the computation time of reading data into memory and estimating an ordinary least squares regression. Tests are all conducted on a system with 1GB of RAM, with no other processes running. In each case, tests are conducted up to the point at which there is insufficient RAM to open the data, this precluding the estimation of standard regression models. Beyond this point, it is still feasible to estimate parameters using Cumulative Least Squares.

variables ranging from 5 (panel (a)), to 50 (panel (d)). Processing time in seconds is documented on the vertical axis of each plot, and the total number of observations in thousands is documented on the horizontal access. Times for standard regression software are presented as hollow squares with dashed lines, while times for cumulative algorithms are presented as hollow circles connected by a solid black line. Each point refers to a specific simulated dataset and the time it takes to estimate parameters, standard errors, and other regression statistics with this data. In this Figure, in each case where cumulative algorithms are used the block size is arbitrarily chosen to contain 10% of the total number of observations.

Across all panels we observe, unsurprisingly, that as the total number of observation grows for a fixed K, processing time increases. For cumulative algorithms, this processing time increases approximately linearly. For example, for the case where K=5, regressions with 5, 10, 15 and 20 million observations take

approximately 30, 60, 90 and 120 seconds to run. This is observed in all panels. Similar linear behaviour is observed in standard regression software when the observation numbers are moderate compared to the total RAM available. However, the linear relationship breaks down and processing times become considerably slower from around the time that the total number of observations approaches around 50% of the memory capacity of the computer.[11] This implies that at relatively small numbers of observations compared to a computer's available memory, the processing time of cumulative procedures are similar to that of standard non-cumulative procedures, however cumulative procedures then rapidly become 2 to 3 times fasted than non-cumulative counterparts. At some point, when the number of observations grows beyond the capacity of the RAM, non-cumulative procedures become infeasible to estimate, while the processing time of cumulative procedures continue to scale linearly indefinitely. If similar tests are run using multiple processor versions of software, similar patterns are observed (Appendix Figure A1).

Results from Figure 1 are based on a block size N_j which is arbitrarily chosen as $N_j = N/10$. If data is very large, blocks of this size will also imply that individual blocks of data cannot fit in memory. In Figure 2 we document processing times of cumulative least squares procedures where the block size is varied from 1% of data up to 50% of data (using sizes of greater than 50% of data is not sensible, as one block will be larger than the other).[12] Once again, we document times across a range of K (panels), and N (horizontal axes). Each point refers to the time for a single regression. We observe that across all cases examined, in general smaller block sizes are marginally faster.

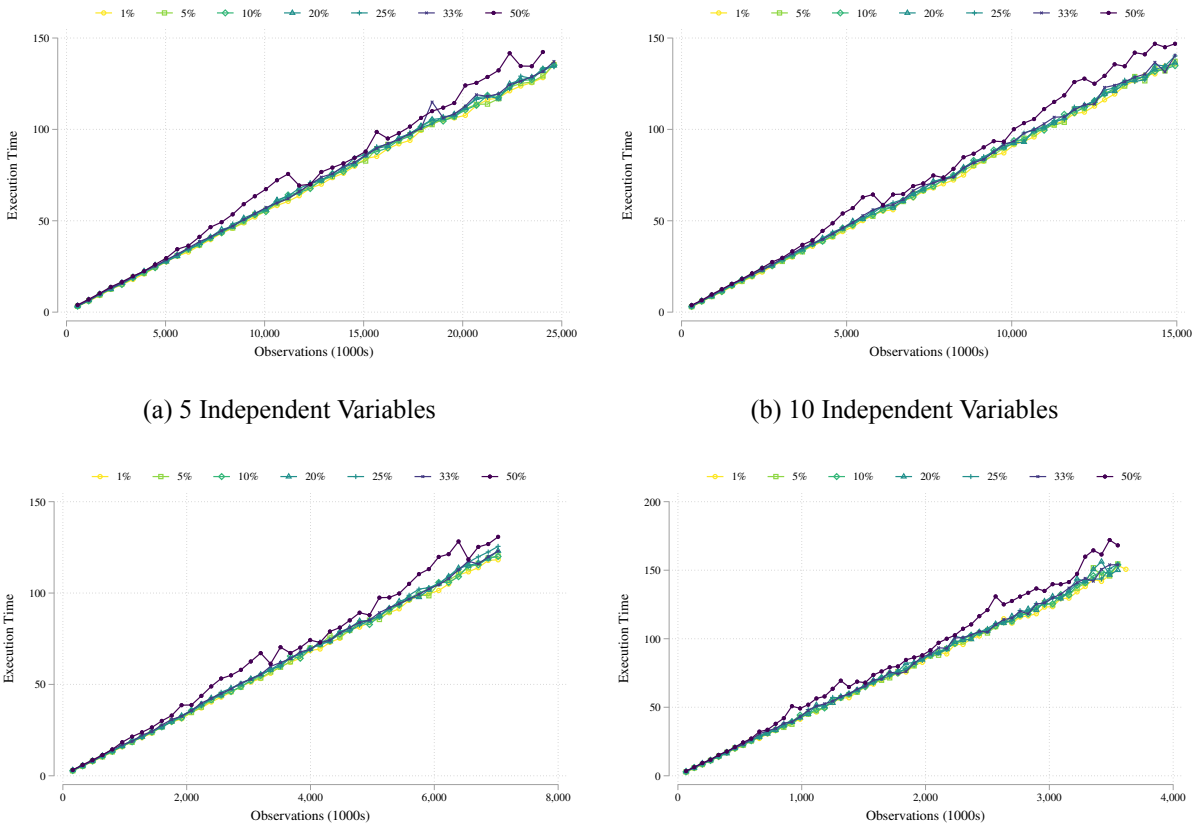
Figure 3 documents the ratio of computation times from Stata's native regress command compared to cumulative least squares, where cumulative least squares is implemented with the same range of block sizes displayed in Figure 2. Values of less than 1 imply that standard (non-cumulative) estimation procedures are faster than cumulative procedures, while values greater than 1 imply that cumulative procedures are faster than non-cumulative procedures. In line with the substantial increase in non-cumulative procedures documented in Figure 1, we observe a sharp improvement in the ratio at around 50% of the theoretical maximum memory. In this particular implementation, when the smallest block size is used (1% of N), the ratio is consistently greater than 1.

These results may suggest that the optimal procedure is thus to choose a block size as small as possible. All results in this paper hold for block sizes as small as $N_j = 1$, and even in cases where $N_j = N/100$,

[11] In principle, a computer with 1GB of RAM contains $1\times10^9 \times \frac{1024^3}{1000^3}$ bytes of memory. A given line of data in our simulations consists of K double precision variables which each occupy 8 bytes of memory ($K-1$ independent variables and the dependent variable). Thus, one can calculate the theoretical maximum number of observations which could be held in memory as $\frac{1\times10^9}{K\times8} \times \frac{1024^3}{1000^3}$. In Figure 1 we observe that the kink in processing times for Stata's regress command occurs at around 12,000,000 observations, or around 44% of the computer's theoretical maximum observations.

[12] These are essentially profiles of a surface where the block size is varied continously. The entire surface is plotted in Appendix Figure A2.

Figure 2: **Execution Time of Cumulative Least Squares by Block Size**



(a) 5 Independent Variables

(b) 10 Independent Variables

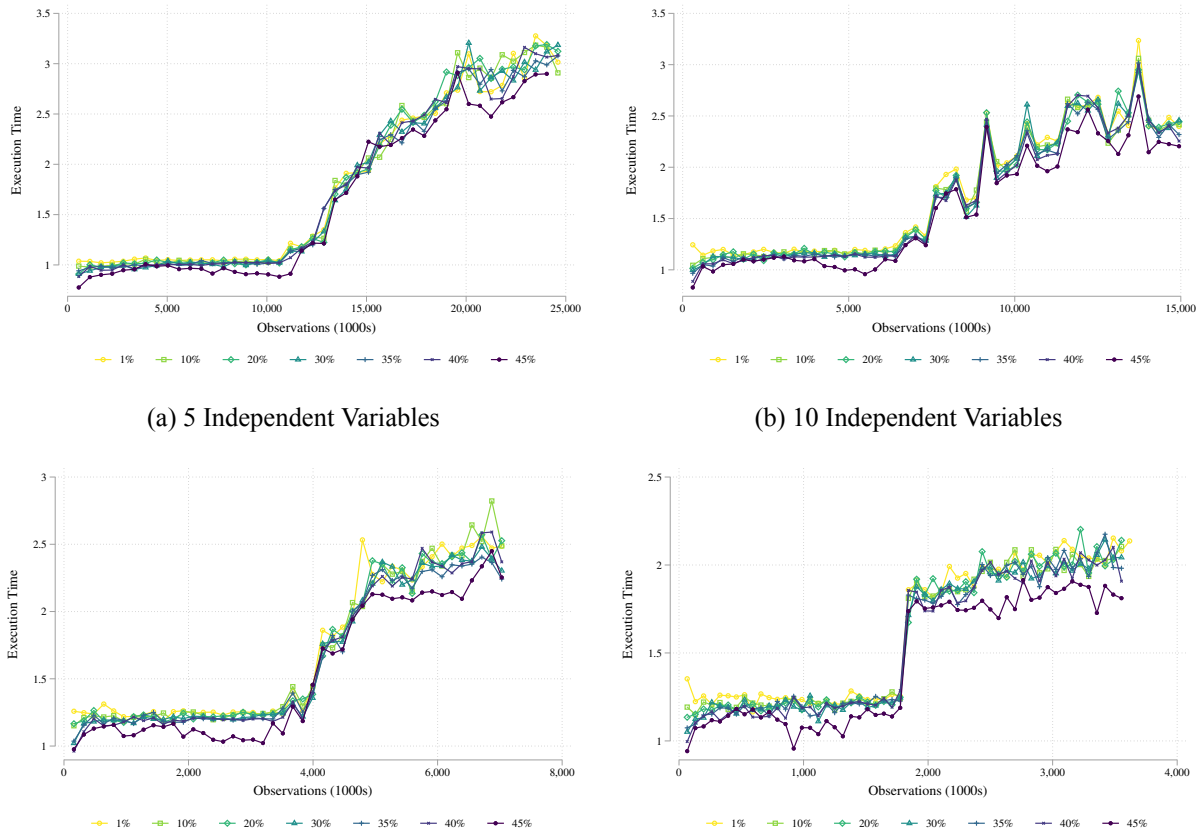
(c) 20 Independent Variables

(d) 50 Independent Variables

Notes: All times refer to the computation time of reading data into memory and estimating a cumulative least squares regression. Tests are all conducted on a system with 1GB of RAM, with no other processes running. Block sizes as a proportion of the total observation numbers are indicated in the figure legend.

the block size considered exceeds 1. In Figure 4 we consider execution times for a particular simulated dataset (N=25,000,000, $K = 5$), however here allowing block sizes to fall to their smallest possible value. A logarithmic scale is used on the horizontal scale allowing black sizes to vary from 1 to 12.5 million observations (50% of the total observations). Panel (a) uses an identical 1GB server as that used in tests above, while panel (b) documents the same times on a computer where memory limits do not bind. In this case we observe that the optimal block size *is not* the smallest possible size ($N = 1$), but rather follows the Goldilocks principle laid out in Section 3. Clearly, block sizes that are so large that memory constraints begin to bind with N_j observations should be avoided, however, a very small block size is also sub-optimal, given that this requires the accumulation of many $X^{j\prime}X^j$ and $X^{j\prime}Y^j$ matrices. Where memory limits do not bind sharply (panel (b)) one may wish to work with slightly larger block sizes to avoid sub-optimal behaviour observed with very small block sizes, as provided extreme regions are avoided, the practical

Figure 3: **Relative Time of Cumulative Least Squares Versus Standard Implementation by Block Size**



(a) 5 Independent Variables

(b) 10 Independent Variables

(c) 20 Independent Variables

(d) 50 Independent Variables

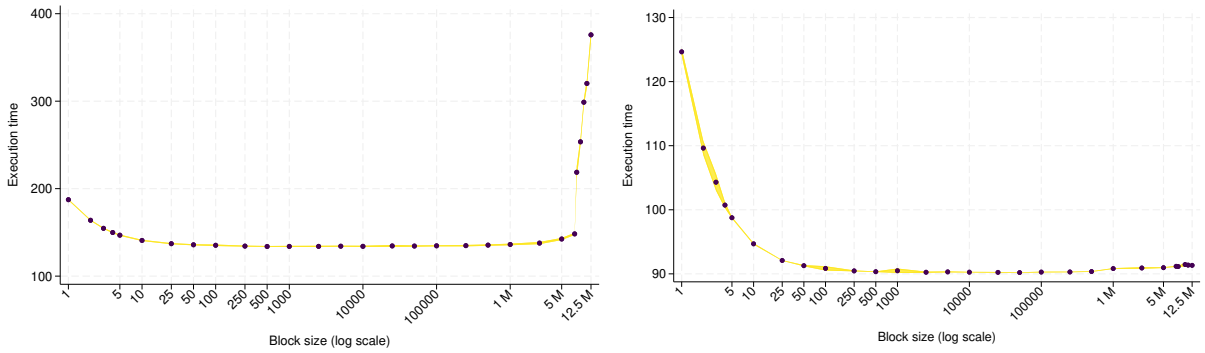
Notes: Each point presents the ratio of computation times of Stata's native regression command to cumulative least squares. Values less than 1 imply non-cumulative procedures are faster than cumulative procedures, and vice versa for values greater than 1. All times refer to the computation time of reading data into memory and estimating a cumulative least squares regression. Tests are all conducted on a system with 1GB of RAM, with no other processes running. Block sizes as a proportion of the total observation numbers are indicated in the figure legend.

choice of block appears to be of second order importance.

4.2 An Empirical Example

We document the performance of cumulative algorithms and their non-cumulative counterparts on a real empirical example. This empirical example is based on a large sample of microdata, following Aaronson et al. (2020). Aaronson et al. estimate the impact of fertilty on mother's labour supply using data over 2 centuries from censuses and demographic surveys. We follow Aaronson et al. (2020) in downloading data from IPUMS and the Demographic and Health Surveys resulting in 51,449,770 observations covering 106 countries, with observations drawn from 434 country by year cells. Data covers years 1787-2015, and

Figure 4: Optimal Block Size and Available Memory



(a) Memory Limits Binding

(b) No Binding Memory Limits

Notes: Total time taken to estimate a regression with 5 independent variables and 25 million observations is documented. Varying block sizes are used, as plotted on the horizontal axes, and total time is plotted on the vertical axis. The left-hand panel is estimated on a PC with 32GB of RAM, 8 cores and Intel i7 processors. The right-hand panel is estimated on a server with 1GB of RAM, 4 cores, and an Intel i7 processor. In each case, estimates are generated 50 times for each block size and average times are plotted as circles. The 95% confidence interval of these estimation times are plotted in yellow.

measures women's labour force participation, total fertility, and a number of other mother-level covariates. In Appendix C we provide summary statistics as well as a graph documenting the years covered in data and the graph documenting the countries covered and the number of observations in each (Figures A3 and A4).

This example is well-suited to our setting because it allows us to document the relative performance of a number of different estimation and inference procedures. Specifically, two models are considered, and these are estimated in a number of ways. A first model is simple (weighted) ordinary least squares, where each woman's labour for participation measure is regressed on her total fertility. We estimate:

$$\text{Participation}_{ict} = \beta_0 + \beta_1 \text{Fertility}_{ict} + X'_{ict}\beta + \phi_{c\times t} + \varepsilon_{ict}, \tag{13}$$

for individual i in country c observed in year t, where country by year fixed effects are indicated as ϕ, and covariates X_{ict} are those indicated by Aaronson et al., namely each women's age, age at first birth, and first born child's sex. Second, we estimate an IV model, where in the first stage, a measure of fertility (specifically whether a woman has a third child) is regressed on an indicator of a woman having second birth twins, and then in a second stage labour force participation is regressed on instrumented fertility:

$$\begin{aligned} \text{Fertilty 3}_{ict} &= \pi_0 + \pi_1 \text{Twin 2}_{ict} + X'_{ict}\Pi + \phi_{c\times t} + \nu_{ict} \\ \text{Participation}_{ict} &= \gamma_0 + \gamma_1 \widehat{\text{Fertility 3}}_{ict} + X'_{ict}\Gamma + \phi_{c\times t} + \eta_{ict}. \end{aligned} \tag{14}$$

All other details follow those laid out in (13), and replicate models proposed by Aaronson et al. (2020).[13] This IV strategy follows a long tradition, starting with Rosenzweig and Wolpin (1980), of seeking to draw conditionally exogenous variation in fertility owing twin births (see Bhalotra and Clarke (2023) for a recent overview).

In this context, we are interested in documenting the processing times of IV and OLS estimation of coefficients and standard errors with a number of specific estimation procedures. This includes IV and OLS models where fixed effects are directly estimated as well as estimation by fixed effects estimators where fixed effects are concentrated out resulting in estimates only of coefficients on time-varying variables. We also consider a number of alternative inference procedures; namely, firstly assuming homoscedasticity, then clustering standard errors by country$\times$year, both analytically, and with a clustered bootstrap. We report the processing time of cumulative algorithms written by us to implement the procedures we lay out above compared to commercially produced (non-cumulative) algorithms written in Stata (version 18). We also consider alternative non-commercial (non-cumulative) algorithms which implement potentially more efficient fixed effect procedures, namely a more rapid implementation of the within transformation described by Gaure (2013), Guimarães and Portugal (2010), Correia (2015), implemented by Correia (2015). All processing times are measured in minutes and include the time of reading data, estimating the regression and producing output, and are estimated under controlled conditions on a a server with fixed characteristics which are varied across tests. Each procedure is estimated 10 times, with average processing times reported.

Results are displayed in Table 1. Each cell provides the average processing time for a particular estimation procedure in a particular computational environment. Estimation procedures are listed in rows, and computational resources are listed in columns. In Panel A we document times corresponding to OLS (13), and in Panel B we document times corresponding to IV (14). Within panels, we first present processing times for cumulative algorithms, and below this, standard regression or IV regression implementations. We replicate each process for a range of computational systems. These are all commercially available dedicated private servers with virtual memory limits (RAM) listed in column headers. Although each column principally changes the quantity of RAM available on the server, a number of other more minor changes may occur on the server when changing configurations. For this reason, in Panel C we provide a system benchmark which shows the server's performance on a standard numerical test.[14]

Considering the behaviour of cumulative algorithms in OLS, we see that irrespective of the computer's memory, the processing time is very similar, ranging from an average of 4.0 to 4.3 minutes when estimation is conducted by within-transformed variables as in (9). This stability across systems is precisely the value

[13] All results are replicated exactly.

[14] Specifically, the system benchmark consists of counting the number of times that the computer is capable of calculating all the primes up to 10,000 in a 10 second span. In this case, higher values of the system benchmark imply that the computer is faster.

Table 1: Execution Time Across Models in a Labour Market Participation Example: OLS and IV

	Memory Constraints					
	1 GB	2 GB	4 GB	8 GB	16 GB	32 GB
Panel A: OLS						
Cumulative Procedures						
Within-transformation	4.060	4.135	4.084	4.111	4.316	4.108
Within, Clustered	7.979	8.053	7.929	7.994	8.418	7.929
Within, Clustered Bootstrap	4.086	4.133	4.090	4.151	4.303	4.084
Within, Clustered Bootstrap (s)	3.684	3.745	3.727	3.798	3.930	3.715
Full Estimation	4.016	4.010	3.989	4.046	4.197	3.951
Standard Procedures						
Within-transformation (areg)	–	–	9.162	8.669	8.311	8.497
Within, Clustered	–	–	12.533	12.034	11.241	11.333
Full Estimation	–	–	10.436	10.147	10.224	10.054
Within-transformation (hdfe)	–	–	–	–	3.605	3.476
Within, Clustered (hdfe)	–	–	–	–	3.794	3.554
Panel B: IV						
Cumulative Procedures						
Within-transformation	4.600	4.608	4.600	4.609	4.774	4.589
Within, Clustered	8.575	8.653	8.609	8.724	8.968	8.636
Within, Clustered Bootstrap	4.428	4.449	4.435	4.429	4.600	4.383
Winthin, Clustered Bootstrap (s)	4.267	4.293	4.281	4.278	4.467	4.271
Full Estimation	4.362	4.379	4.400	4.350	4.577	4.329
Standard Procedures						
Within-transformation (xtivreg)	–	–	–	–	–	6.629
Within, Clustered	–	–	–	–	–	15.958
Full Estimation	–	–	–	–	–	226.181
Within-transformation (hdfe)	–	–	–	–	–	5.996
Within, Clustered (hdfe)	–	–	–	–	–	8.597
Panel C: System Benchmark	768.1	764.9	768.1	769.9	772.7	778.5

Notes: Each cell reports average processing time in minutes of a particular estimation procedure based on the memory constraints listed in column headers. All averages are taken over 10 estimations. Cells are coloured gray when estimation cannot occur due to memory limits. All methods are estimated in Stata 18 SE. For clustered bootstrap errors, (s) stands for "sorted", implying that the database is sorted by clusters prior to processing. System benchmark is a standard test of processor capacity run on the operating system to provide a baseline comparison of server performance across columns.

of cumulative regression procedures. Here, whether one has available a system with substantial memory (32 GB) or very little memory (1 GB), there is no change in performance. The case of standard regression is of course different. In systems with small memory capacity it is simply infeasible to load data and estimate parameters. These cells are coloured gray. Initially, when it is feasible to load data (a system with 4 GB), the standard implementation is substantially slower than its cumulative counterpart (9.1 minutes versus 4.1 minutes). Moving to larger memory capacities reduces the time of processing of within transformed

models slightly (to around 8.3 to 8.6 minutes), but given the efficiency with which cumulative routines can conduct fixed effect procedures (Section 2.3), standard implementations do not approach the performance of cumulative algorithms. Alternative routines for dealing with fixed effects are observed to be slightly faster when they are feasible to be estimated, as observed when within-transformed models from cumulative algorithms are compared with hdfe implementations. However, such models are infeasible in a range of cases where lower memory limits are binding, and similarly cannot return full parameter vectors.

When we wish to report full parameter vectors ("Full Estimation"), cumulative algorithms outperform comparison estimators. Across system types, cumulative procedures require between 3.9 to 4.2 minutes for estimation, while similar procedures in non-cumulative models require around 10 minutes. It is noteworthy that in the case of cumulative algorithms, full estimation including fixed effects is marginally faster than within-transformed versions, and this owes to the fact that within transformations require conducting group-level analyses, while models to return full fixed effects do not.

Turning to inference, we observe that when calculating clustered standard errors analytically, the processing time of cumulative algorithms approximately doubles. This owes to the procedure laid out in Section 2.4 where the calculation of standard errors requires estimates of $\widehat{u}$, and hence requires reading data two times. However, One particular advantage of fixed effect estimation is that if clustered standard errors are desired, running block bootstraps is virtually costless. In Table 1 we run 10,000 bootstraps following the procedure laid out in Section 2.4.3, seeing that this adds nearly no processing time (row 1 compared to row 3 of Table 1). We do not display the comparison for standard clustered bootstrap processing times in the table in a *non-cumulative* process, simply because this procedure increases linearly in the number of bootstraps, and is many orders of magnitude slower than cumulative procedures. In general, all group-level processing in Table 1 occurs when data is sorted in any order, however if data is indeed sorted by group prior to processing, estimation time further falls given that group-level processing of data can occur more rapidly (row 4 of panel A).

In the case of IV models, results are even more stark given that non-cumulative calculations require housing larger matrices in memory, and make processing infeasible at a broader range of memory restrictions. Indeed, in this case, even with data with only approximately 50 million observations, processing in non-cumulative routines becomes feasible at 32 GB of memory, but not before. We again see a number of key take-aways from Panel B of Table 1. These are, firstly, that cumulative routines open up feasibility where previously estimation could not occur. Secondly, even where processing can occur, cumulative algorithms are generally faster than non-cumulative procedures. In some cases, presumably where data approaches the memory limits of the computer, comparisons suggest that even where feasible, non-cumulative methods may be considerably slower than cumulative counterparts. This is especially clear in the comparison of IV

models where all fixed effects are estimated (226 minutes) to a similar procedure in cumulative 2SLS (4.3 minutes).

It is important to note that this example should be conceived as simply illustrative of the fact that along with the conceptual interest of cumulative procedures, they do have practical implications too. Our implementations are unlikely to compare to commercial implementations in terms of the efficiency of every internal process, so could be conceived as lower bounds of the performance in this particular setting. In other languages and on other specific computational environments results will also vary. What's more, while this data has a reasonable number of observations (around 50 million), estimation is feasible with as little as 4GB of memory depending on the estimation procedure. However, in cases with much larger data, such memory limits will be far more binding, suggesting that the feasibility benefit of cumulative algorithms may be more important.

5 Discussion and Conclusions

In this paper we show that regressions can be estimated row-wise, without ever requiring all of the information from a dataset at a given moment of time. In the limit, we show that regressions can be estimated in a simple fashion where only a single line of data is read at a time and then forgotten, with only a small number of low dimensional matrices needing to be updated over time.

Despite the ubiquity of procedures which work in a column-wise fashion in econometrics, based on results known since the seminal papers of Frisch and Waugh (1933), Lovell (1963), to our knowledge these row-wise results have hereto forth not been documented. We show that these results hold for a broad class of regression models including OLS, IV, fixed effect, and regularised regression models such as Ridge and LASSO, and that the logic of these results holds both for both least squares and other M-class estimators such as probit and logit models. These results are not approximations, providing exact calculations of regression coefficients, and can similarly generate standard errors and other regression statistics such as goodness of fit measures exactly. Turning to inference, we show that these methods apply for both homoescedastic error assumptions, as well as for heteroscedasticity- and cluster-robust variance estimators. We additionally document that certain bootstrap procedures and the definition of tuning parameters can be substantially more efficient with applications of the results from this paper.

As well as the theoretical interest of understanding the mechanics of frequently used regression models, these results have a large number of practical uses. In both simulated and real data we show that these results imply that models which cannot be estimated on certain computers using standard commercial implementations of regression software can be estimated using the same programs but with our algorithms. What's

more, even in cases where a computer's memory does not limit data from being opened, we show that our algorithms can at times offer non-trivial speed ups over standard software. In some sense these results could be cast as democratising the processing of big data via regression, as, provided that a sufficiently large hard disk is available, one could process extremely large datasets with very low memory requirements. Indeed, there is no reason why these results could not be applied to data stored remotely, or on the web, implying that it would not be necessary to have access to super computers to process data of any size.

The results in this paper are likely lower bounds of the true performance of these algorithms. We have implemented the results in this paper in a high level matrix processing language, and comparisons are made to programs written largely in faster low-level languages. What's more, the procedures we use here process all data in a naive sequential fashion. The results from this study make clear that regression is an embarrassingly parallelisable task, and so if data is stored or broken down into various chunks in different files, processing times likely also scale approximately linearly in parallel processes. All told, these results suggest that transposing the ideas of Frisch-Waugh-Lovell to process regressions in a row-by-row rather than column-by-column fashion provides both an interesting theoretical proposition, as well as useful practical applications.

References

Aaronson, D., R. Dehejia, A. Jordan, C. Pop-Eleches, C. Samii, and K. Schulze (2020, 08). The Effect of Fertility on Mothers' Labor Supply over the Last Two Centuries. *The Economic Journal 131*(633), 1–32.

Abadie, A. (2003, April). Semiparametric instrumental variable estimation of treatment response models. *Journal of Econometrics 113*(2), 231–263.

Abadie, A., S. Athey, G. W. Imbens, and J. M. Wooldridge (2020). Sampling-Based versus Design-Based Uncertainty in Regression Analysis. *Econometrica 88*(1), 265–296.

Angrist, J. D. (1998). Estimating the Labor Market Impact of Voluntary Military Service Using Social Security Data on Military Applicants. *Econometrica 66*(2), 249–288.

Baltagi, B. H. (2001). *Econometric Analysis of Panel Data* (2nd ed.). Chichester: John Wiley and Sons.

Bhalotra, S. and D. Clarke (2023). *Analysis of Twins*, pp. 1–37. Springer International Publishing.

Cameron, A. C., J. B. Gelbach, and D. L. Miller (2008, August). Bootstrap-Based Improvements for Inference with Clustered Errors. *The Review of Economics and Statistics 90*(3), 414–427.

Correia, S. (2015, March). HDFE: Stata module to partial out variables with respect to a set of fixed effects. Statistical Software Components, Boston College Department of Economics.

Frisch, R. and F. V. Waugh (1933). Partial Time Regressions as Compared with Individual Trends. *Econometrica 1*(4), 387–401.

Fu, W. (1998). Penalized Regressions: The Bridge vs. the Lasso. *Journal of Computational and Graphical Statistics 7*(3), 397–416.

Gaure, S. (2013). OLS with multiple high dimensional category variables. *Computational Statistics & Data Analysis 66*, 8–18.

Gelbach, J. B. (2016). When Do Covariates Matter? And Which Ones, and How Much? *Journal of Labor Economics 34*(2), 509–543.

Green, P. J. (1984). Iteratively Reweighted Least Squares for Maximum Likelihood Estimation, and some Robust and Resistant Alternatives. *Journal of the Royal Statistical Society. Series B (Methodological) 46*(2), 149–192.

Guimarães, P. and P. Portugal (2010, December). A simple feasible procedure to fit models with high-dimensional fixed effects. *Stata Journal 10*(4), 628–649.

Hager, W. W. (1989, 6). Updating the Inverse of a Matrix. *SIAM Review 31*(2), 221–239.

Lovell, M. C. (1963). Seasonal Adjustment of Economic Time Series and Multiple Regression Analysis. *Journal of the American Statistical Association 58*(304), 993–1010.

MacKinnon, J. G. (2023a). Fast cluster bootstrap methods for linear regression models. *Econometrics and Statistics 26*, 52–71.

MacKinnon, J. G. (2023b). Using large samples in econometrics. *Journal of Econometrics 235*(2), 922–926.

Miller, K. S. (1981, 3). On the Inverse of the Sum of Matrices. *Mathematics Magazine 54*(2), 67–72.

Nelder, J. A. and R. W. M. Wedderburn (1972). Generalized linear models. *Journal of the Royal Statistical Society. Series A (General) 135*(3), 370–384.

Roodman, D., M. Ørregaard Nielsen, J. G. MacKinnon, and M. D. Webb (2019). Fast and wild: Bootstrap inference in Stata using boottest. *The Stata Journal 19*(1), 4–60.

Rosenzweig, M. R. and K. I. Wolpin (1980). Testing the Quantity-Quality Fertility Model: The Use of Twins as a Natural Experiment. *Econometrica 48*(1), 227–240.

Solon, G., S. J. Haider, and J. M. Wooldridge (2015). What Are We Weighting For? *Journal of Human Resources 50*(2), 301–316.

Stefanski, L. A. and D. D. Boos (2002). The Calculus of M-Estimation. *The American Statistician 56*(1), 29–38.

Słoczyński, T. (2022, 05). Interpreting OLS Estimands When Treatment Effects Are Heterogeneous: Smaller Groups Get Larger Weights. *The Review of Economics and Statistics 104*(3), 501–509.

Varian, H. R. (2014, May). Big Data: New Tricks for Econometrics. *Journal of Economic Perspectives 28*(2), 3–28.

Woodbury, M. A. (1950). Inverting modified matrices. Memorandum Report 42, Statistical Research Group, Princeton University, Princeton, NJ.

Wooldridge, J. M. (2021, August 17). Two-Way Fixed Effects, the Two-Way Mundlak Regression, and Difference-in-Differences Estimators. Technical report, Social Science Research Network. Available at SSRN: `https://ssrn.com/abstract=3906345` or `http://dx.doi.org/10.2139/ssrn.3906345`.

Wu, Y. and L. Wang (2020). A Survey of Tuning Parameter Selection for High-Dimensional Regression. *Annual Review of Statistics and Its Application 7*(1), 209–226.

Appendices for: Frisch-Waugh-Lovell′

Damian Clarke, Nicolás Paris & Benjamina Villena-Rodán

Not for print.

A A Simple Visualisation in Matrix Form

Consider a simple illustration of the generation of $X'X$ based on a case where X is a matrix consisting of $N = 4$ observations and $K = 3$ independent variables. For ease of visualisation, we will denote X as follows:

$$\begin{pmatrix} x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,1} & x_{3,2} & x_{3,3} \\ x_{4,1} & x_{4,2} & x_{4,3} \end{pmatrix}$$

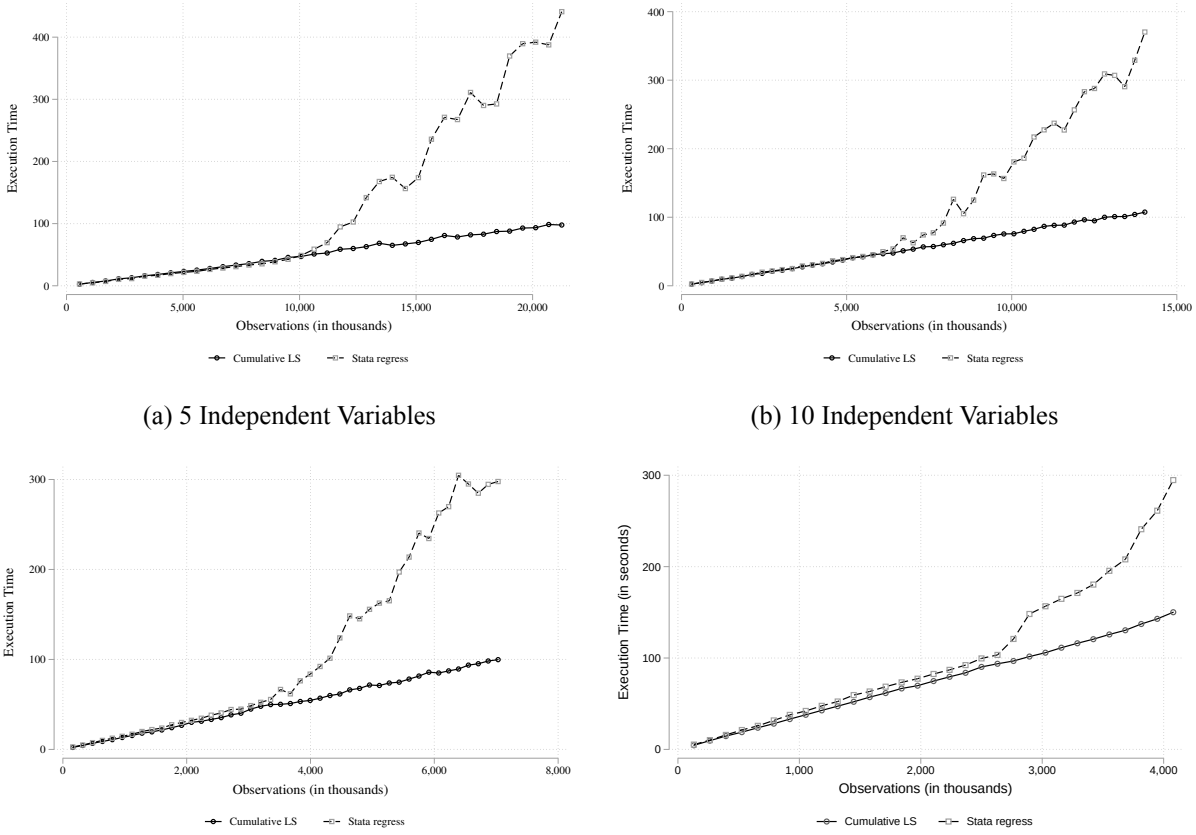
where all realisations of independent variables for observation $i = 1$ are coloured in blue, and for observation $i = 4$ are coloured in red. Now note that $X'X$ can be written in extensive form as below:

$$\begin{aligned} X'X \equiv & \begin{pmatrix} x_{1,1} & x_{2,1} & x_{3,1} & x_{4,1} \\ x_{1,2} & x_{2,2} & x_{3,2} & x_{4,2} \\ x_{1,3} & x_{2,3} & x_{3,3} & x_{4,3} \end{pmatrix} \begin{pmatrix} x_{1,1} & x_{1,2} & x_{1,3} \\ x_{2,1} & x_{2,2} & x_{2,3} \\ x_{3,1} & x_{3,2} & x_{3,3} \\ x_{4,1} & x_{4,2} & x_{4,3} \end{pmatrix} \\ = & \begin{pmatrix} x_{1,1}^2 + x_{2,1}^2 + x_{3,1}^2 + x_{4,1}^2 & x_{1,1}x_{1,2} + x_{2,1}x_{2,2} + x_{3,1}x_{3,2} + x_{4,1}x_{4,2} & x_{1,1}x_{1,3} + x_{2,1}x_{2,3} + x_{3,1}x_{3,3} + x_{4,1}x_{4,3} \\ x_{1,2}x_{1,1} + x_{2,2}x_{2,1} + x_{3,2}x_{3,1} + x_{4,2}x_{4,1} & x_{1,2}^2 + x_{2,2}^2 + x_{3,2}^2 + x_{4,2}^2 & x_{1,3}x_{1,1} + x_{2,3}x_{2,1} + x_{3,3}x_{3,1} + x_{4,3}x_{4,1} \\ x_{1,3}x_{1,1} + x_{2,3}x_{2,1} + x_{3,3}x_{3,1} + x_{4,3}x_{4,1} & x_{1,3}x_{1,2} + x_{2,3}x_{2,2} + x_{3,3}x_{3,2} + x_{4,3}x_{4,2} & x_{1,3}^2 + x_{2,3}^2 + x_{3,3}^2 + x_{4,3}^2 \end{pmatrix} \\ = & \begin{pmatrix} x_{1,1}^2 & x_{1,1}x_{1,2} & x_{1,1}x_{1,3} \\ x_{1,2}x_{1,1} & x_{1,3}x_{1,1} & x_{1,3}x_{1,1} \\ x_{1,3}x_{1,1} & x_{1,3}x_{1,2} & x_{1,3}^2 \end{pmatrix} + \begin{pmatrix} x_{2,1}^2 & x_{2,1}x_{2,2} & x_{2,1}x_{2,3} \\ x_{2,2}x_{2,1} & x_{2,2}^2 & x_{2,3}x_{2,1} \\ x_{2,3}x_{2,1} & x_{2,3}x_{2,2} & x_{2,3}^2 \end{pmatrix} + \cdots + \begin{pmatrix} x_{4,1}^2 & x_{4,1}x_{4,2} & x_{4,1}x_{4,3} \\ x_{4,2}x_{4,1} & x_{4,2}^2 & x_{4,3}x_{4,1} \\ x_{4,3}x_{4,1} & x_{4,3}x_{4,2} & x_{4,3}^2 \end{pmatrix}. \end{aligned} \tag{15}$$

The key takeaway here is that one can simply arrive to the $K \times K$ matrix $X'X$ by calculating 4 $K \times K$ matrices based on cross-products for each observation (ie working an observation at a time), and finally summing across these matrices (as in (15)).

B Appendix Figures and Tables

Figure A1: Sample size and execution time of standard versus cumulative (Multiple Cores)



(a) 5 Independent Variables

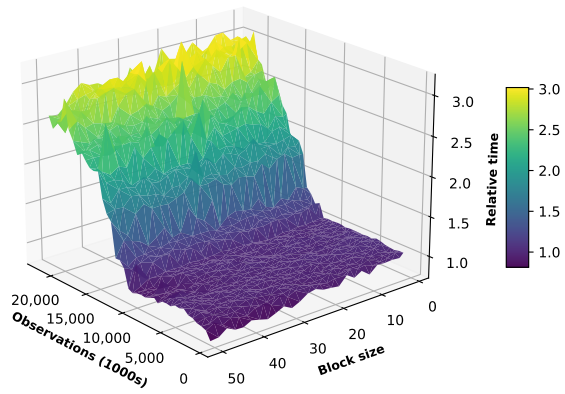
(b) 10 Independent Variables

(c) 20 Independent Variables

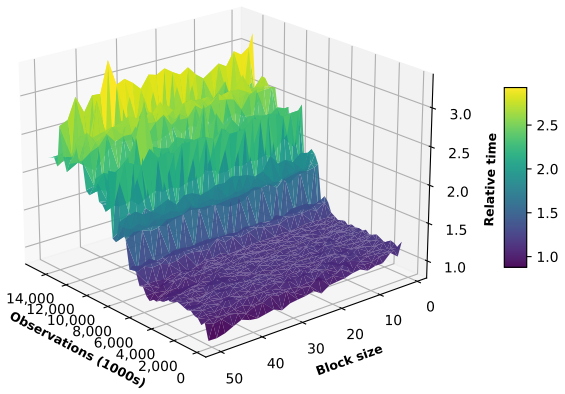
(d) 50 Independent Variables

Notes: All times refer to the computation time of reading data into memory and estimating an ordinary least squares regression. Tests are all conducted on a system with 1GB of RAM, with no other processes running, using Stata MP (2 cores). In each case, tests are conducted up to the point at which there is insufficient RAM to open the data, thus precluding the estimation of standard regression models. Beyond this point, it is still feasible to estimate parameters using Cumulative Least Squares.

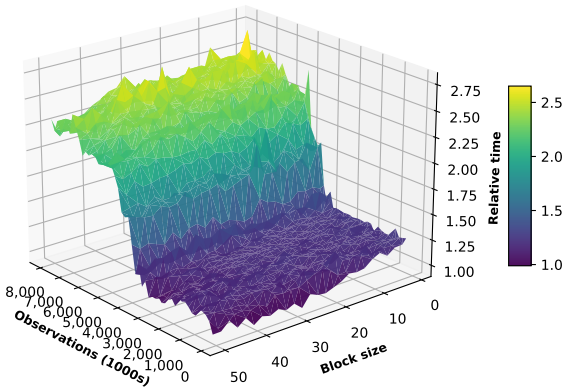
Figure A2: Relative Performance of OLS to CLS by Block Size



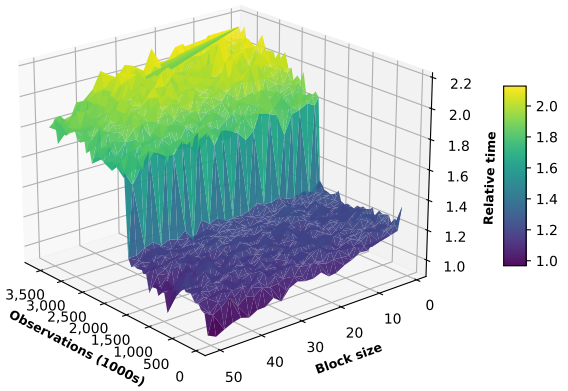
(a) 5 Independent Variables



(b) 10 Independent Variables



(c) 20 Independent Variables



(d) 50 Independent Variables

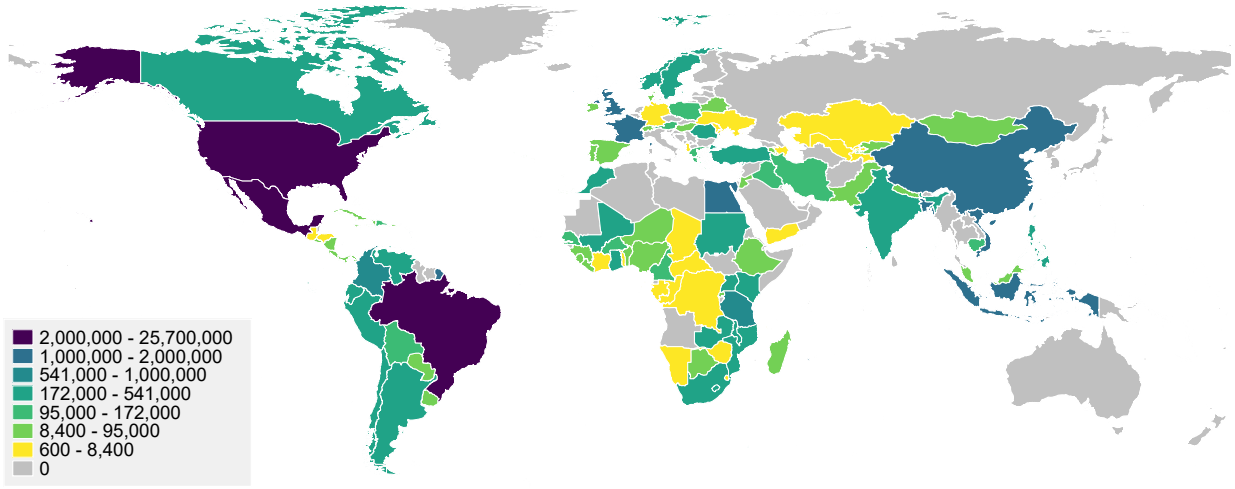
Notes: Surface plots present the ratio of computation times of Stata's native regression command to cumulative least squares. Values less than 1 imply non-cumulative procedures are faster than cumulative procedures, and vice versa for values greater than 1. All times refer to the computational time of reading data into memory and estimating a cumulative least squares regression. Tests are all conducted on a system with 1GB of RAM, with no other processes running. Block sizes indicated on the horizontal axis refer to the proportion of the full dataset (*eg* 50 refers to 2 blocks each covering 50% of the data, 1 refers to 100 blocks each covering 1% of the data).

C Data Appendix

We collate original data from IPUMS and the Demographich and Health Survey (DHS) repositry using all census data and DHS waves described in (Aaronson, Dehejia, Jordan, Pop-Eleches, Samii, and Schulze 2020). This results in 51,449,770 observations drawn from 434 census files for 106 countries covering years 1787 to 2015. The geographical coverage of the data is described in Figure A3, and the temporal coverage is described in Figure A4.

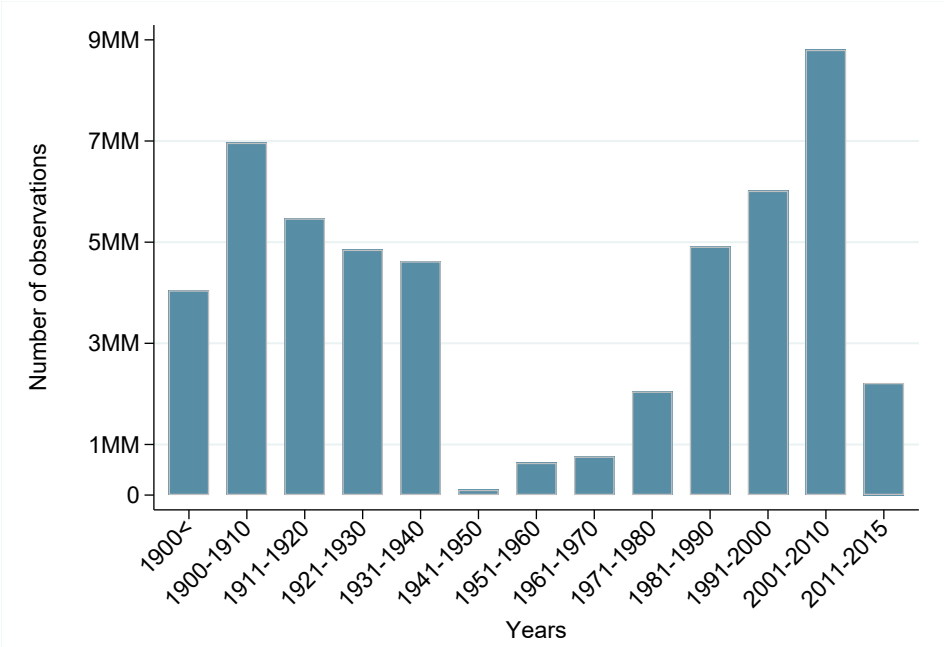
We follow replication materials of (Aaronson, Dehejia, Jordan, Pop-Eleches, Samii, and Schulze 2020) to generate all variables, and replicate their results exactly. We follow their inclusion criteria of working with women aged 21 to 35 who have at least two children, all of whom are 17 or younger. As descriped in Aaronson et al. (2020) families are excluded where information is missing on child gender or mother's age, and mothers are not included in the same if they live in group quarters or give birth before the age of 15. Summary statistics of all data following the processes described in Aaronson et al., (2020) are included below in Table A1.

Figure A3: Number of Observations by Country



Notes: Values plotted refer to the total frequency of observations used in estimating samples. These are pooled across all years. Countries coloured grey have no available microdata on IPUMS or DHS.

Figure A4: Number of Observations by Year



Notes: Values refer to the total frequency of observations used in estimating samples. Years refer to the year the data was collected.

Table A1: Summary Statistics – Principal Variables

VARIABLES	Obs	Mean	Std. Dev.	Min	Max
Labour Force Participation	51,449,770	0.263	0.441	0	1
Covariables					
Fertility	51,449,770	0.525	0.499	0	1
Gender of first child	51,449,770	0.508	0.499	0	1
Mother Age	51,449,770	29.43	3.859	21	35
Mother age at first birth	51,449,770	21.04	3.300	15	35
Weights	51,449,770	1	0.390	0.0007	341.66
Years	51,449,770	1954.95	44.169	1787	2015
Instrument					
Twin	51,449,770	0.0105	0.102	0	1

Notes: Summary statistics are displayed based on all observations and data cleaning procedures described in Aaronson et al. (2020). Sample consists of women giving birth at the age of 15 or above, and ages 21 to 35 at the time of data collection. All selection criteria follows Aaronson et al. (2020).

D An Updating Estimation Procedure

Throughout the paper we work with a cumulative procedure in which matrices $X'X$ and $X'Y$ are accumulated in a step-wise fashion, and estimates $\widehat{\beta}_{OLS}$ (or similar for other estimation methodologies) are generated only after data is read in its entirety. Alternative procedures can be used in which iterations occur over sequential iterations of $\widehat{\beta}_{OLS}$ estimates themselves, though these are less efficient than the cumulative procedures laid out in the body of the paper.

To see this, we will use identical notation to that laid out in the paper. Suppose we wish to estimate a regression between a dependent variable Y and a set of K covariates $X_1, X_2, ..., X_K$. The database that can be partitioned into J samples. The whole sample size of the database is N, but computing an OLS regression with all the data is unfeasible due to memory constrains. As an alternative procedure, we could run a regression with the sample $j = 1$ and update the result with the other $J - 1$ samples. To fix ideas, suppose we have a samples 1 and 2 and we want to compute an OLS estimator for the whole sample denoted by the subindex $1 \sim 2$. Hence,

$$\begin{aligned}\hat{\beta}_{1\sim 2} &= \left(\begin{pmatrix} X_1' & X_2' \end{pmatrix} \begin{pmatrix} X_1 \\ X_2 \end{pmatrix} \right)^{-1} \begin{pmatrix} X_1' & X_2' \end{pmatrix} \begin{pmatrix} Y_1 \\ Y_2 \end{pmatrix} = (X_1'X_1 + X_2'X_2)^{-1}(X_1'Y_1 + X_2'Y_2) \\ &= (\Sigma_1 + \Sigma_2)^{-1}(\Upsilon_1 + \Upsilon_2)\end{aligned}$$

where $X_j'X_j \equiv \Sigma_j$ and $X_j'Y_j \equiv \Upsilon_j$. The challenge is to compute $\hat{\beta}_{1\sim 2}$ using estimates from the two samples separately, so that we can avoid storing very large databases in memory. Trivially, as laid out in the body of the paper, this can be done cumulatively. Alternatively, we can make use of a result of the inverse of the sum of two matrices by Miller (1981). A more general perspective in this theory, including application to linear least squares is Hager (1989). The application of this result can be proven easily.

Lemma 1 *The inverse of the sum of two matrices can be obtained as*

$$(\Sigma_1 + \Sigma_2)^{-1} \equiv \Sigma_{1\sim 2}^{-1} = \Sigma_1^{-1} - \Sigma_1^{-1}\Sigma_2(I + \Sigma_1^{-1}\Sigma_2)^{-1}\Sigma_1^{-1}$$

Proof. Follows as a direct application of Woodbury (1950) matrix inverse lemma ■

Using Lemma 1, we can iterate on $\widehat{\beta}_{OLS}$, without requiring that all of the elements stored in cumulative procedures are accumulated across iterations. We define the factor

$$\Omega_{1\sim 2} \equiv I_K - \Sigma_1^{-1}\Sigma_2 \left(I_K + \Sigma_1^{-1}\Sigma_2\right)^{-1},$$

where I_K stands for an identity matrix of dimension K, the number of regressors in the model. Hence, the

joint Σ matrix can be expressed as

$$\Sigma_{1\sim 2}^{-1} = \Omega_{1\sim 2}\Sigma_1^{-1}$$

Therefore, the joint OLS estimator becomes

$$\widehat{\beta}_{1\sim 2} = \Omega_{1\sim 2}\left(\widehat{\beta}_1 + \Sigma_1^{-1}\Upsilon_2\right). \tag{16}$$

This suggests an iterative procedure for estimating OLS parameters generalising the above result to J blocks, rather than 2 blocks. This is defined as Updated Ordinary Least Squares in Algorithm 6.

Algorithm 6: Updated Ordinary Least Squares

Inputs: Database consisting of (y,X), block size b.
Result: Point estimate $\widehat{\beta}_{OLS}$ (and potentially point estimates updated at each step, $\widehat{\beta}_{1\sim j,OLS}$).

1. Set $i = 1$ and $j = b$. Load into memory partition of data covering y, X in observations $i - j$. ;
2. Calculate Σ_1 and $\widehat{\beta}_1$;
3. If observations i-j contain end of file, set $e = 1$, otherwise, set $e = 0$;

while $e \neq 1$ **do**

4. Replace $i = i + b$ and $j = j + b$. Load into memory partition of data covering Y, X in observations $i - j$. ;
5. Calculate Σ_j and Υ_j. ;
6. Calculate $\Omega_{1\sim j} = I_K - \Sigma_{j-1}^{-1}\Sigma_j\left(I_K + \Sigma_{j-1}^{-1}\Sigma_j\right)$ and $\widehat{\beta}_{1\sim j} = \Omega_{1\sim j}\left(\widehat{\beta}_{j-1} + \Sigma_{j-1}^{-1}\Upsilon_j\right)$;
7. If observations i-j contain end of file, set $e = 1$, otherwise, set $e = 0$;

end

While this procedure allows for the calculation of updated values of $\widehat{\beta}_{1\sim j}$ at each step, and additionally avoids the need of passing $\Upsilon_{1\sim j}$ across steps, each iteration involves one matrix inversion of size K in step 6. Indeed, for any given quantity of variables K and block size b, the cumulative algorithm strictly denominates the updating algorithm in terms of total computations (and hence computation time). This owes to the fact that the same calculations of order $\mathcal{O}(NK^2) + O(NK)$ discussed in Section 3 are required in calculating Σ_j and Υ_j as inputs for (16), strictly *more* elements are required to be summed in iterating on $\Omega_{1\sim j}$ instead of $\Sigma_{1\sim j}$ and $\Upsilon_{1\sim j}$, and additionally, a matrix inversion is required at each step in calculating (16). For this reason, we focus on cumulative least squares algorithms throughout this paper.